

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО



МЕТОДИЧНІ ВКАЗІВКИ
ЩОДО ВИКОНАННЯ ЛАБОРАТОРНИХ РОБІТ
З НАВЧАЛЬНОЇ ДИСЦИПЛІНИ
«ПРОГРАМУВАННЯ»
ДЛЯ СТУДЕНТІВ ДЕННОЇ ФОРМИ НАВЧАННЯ
ЗІ СПЕЦІАЛЬНОСТІ 123 – КОМП'ЮТЕРНА ІНЖЕНЕРІЯ

КРЕМЕНЧУК 2022

Методичні вказівки щодо виконання лабораторних робіт з навчальної дисципліни «Програмування» для студентів денної форми навчання зі спеціальності 123 – «Комп'ютерна інженерія»

Укладачі: к. т. н., доц. Д. Г. Мамчур

Рецензент к. т. н., доц. В. М. Сидоренко

Кафедра комп'ютерної інженерії та електроніки

Затверджено методичною радою Кременчуцького національного університету імені Михайла Остроградського

Протокол №_____від_____2022 р.

Голова методичної ради _____ проф. В. В. Костін

ЗМІСТ

Вступ.....	4
1 Перелік лабораторних робіт.....	7
Лабораторна робота № 1 Програмування лінійних процесів, введення та виведення даних засобами мови С.....	7
Лабораторна робота № 2 Програмування лінійних процесів, введення та виведення даних засобами мови С++.....	20
Лабораторна робота № 3 Функції користувача з параметрами-значеннями засобами мови С/С++.....	29
Лабораторна робота № 4 Програмування розгалужених процесів за допомогою оператора if.....	31
Лабораторна робота № 5 Програмування циклічних процесів мовою С/С++.....	39
Лабораторна робота № 6 Одновимірні масиви.....	46
Лабораторна робота № 7 Багатовимірні масиви.....	53
2 Критерії оцінювання знань студентів.....	60
Список літератури.....	61

ВСТУП

Навчальна дисципліна «Програмування» входить до основних дисциплін підготовки здобувачів освіти зі спеціальності 123 – «Комп'ютерна інженерія».

Мета навчальної дисципліни полягає у ознайомленні студентів з можливостями ЕОМ, як засобу дослідження, автоматизації обробки даних; розвитку алгоритмічного мислення; набутті навичок програмування мовами середнього (C) та високого (C++) рівня; засвоєнні необхідних знань з об'єктно-орієнтованого (ООП) програмування, отримання навичок у створенні та використанні класів, з використанням інкапсуляції даних, механізму наслідування, поліморфізму.

Завданням навчальної дисципліни «Програмування» є засвоєння студентами масиву знань з розробки програмного забезпечення з використанням мов програмування середнього та високого рівня, оволодіння практичними навичками створення застосунків для розв'язання типових задач, опановування базових алгоритмів програмування і т.ін.

Дисципліна забезпечує формування компетенцій та досягнення програмних результатів навчання (ПРН):

ЗК1. Здатність до абстрактного мислення, аналізу та синтезу.

ЗК2. Здатність вчитися і оволодівати сучасними знаннями як в колективі, так і самотійно.

ЗК3. Здатність застосовувати знання у практичних ситуаціях та проводити дослідження на відповідному рівні.

ЗК4. Знання та розуміння предметної області професійної діяльності.

ЗК5. Здатність спілкуватися державною та іноземною мовами як усно, так і письмово.

ЗК7. Вміння виявляти, ставити та вирішувати проблеми.

ЗК11. Здатність приймати обґрунтовані рішення.

ФК2. Здатність використовувати сучасні методи і мови програмування для розроблення алгоритмічного та програмного забезпечення.

ФК3. Здатність створювати системне та прикладне програмне забезпечення комп'ютерних систем та мереж.

ФК6. Здатність проектувати, впроваджувати та обслуговувати комп'ютерні системи та мережі різного виду та призначення.

ФК7. Здатність використовувати та впроваджувати нові технології, включаючи технології розумних, мобільних, зелених і безпечних обчислень, брати участь в модернізації та реконструкції комп'ютерних систем та мереж, різноманітних вбудованих і розподілених додатків, зокрема з метою підвищення їх ефективності.

ФК8. Готовність брати участь у роботах впровадження комп'ютерних систем та мереж, введення їх до експлуатації на об'єктах різного призначення.

ФК10. Здатність здійснювати організацію робочих місць, їхнє технічне оснащення, розміщення комп'ютерного устаткування, використання організаційних, технічних, алгоритмічних та інших методів і засобів захисту інформації.

ФК11. Здатність оформляти отримані робочі результати у вигляді презентацій, науково-технічних звітів.

ФК13. Здатність вирішувати проблеми у галузі комп'ютерних та інформаційних технологій, визначати обмеження цих технологій.

ФК15. Здатність аргументувати вибір методів розв'язування спеціалізованих задач, критично оцінювати отримані результати, обґрунтовувати та захищати прийняті рішення.

Згідно вимогам освітньо-професійної програми студент повинен **знати:**

– принципи програмування, засоби сучасних мов програмування, основні структури даних;

- методи розробки алгоритмів та складання програм на мовах середнього (C) та високого рівнях (C++) програмування, методи відлагодження та редагування програмних кодів;

- основні концепції та ключові моменти ООП – парадигми;

- найпростіші технології проведення експериментів, збирання даних та моделювання в комп'ютерних системах;

уміти:

- визначати для вирішення задачі технологію програмування, мову, систему програмування, інструментальне середовище;

- здійснювати функціональну та об'єктну декомпозицію програми відповідно до обраної технології програмування;

- виконувати розробку коду програми;

- виправляти синтаксичні та семантичні помилки, виконувати рефакторинг коду (налагоджувати та тестувати програму);

- володіти методами та технологіями об'єктно-орієнтованого програмування;

- програмувати динамічні структури даних;

- обробляти виключення;

- системно мислити та застосовувати творчі здібності до формування нових ідей;

- здійснювати пошук інформації в різних джерелах для розв'язання задач;

- ефективно працювати як індивідуально так і у складі команди;

- оцінювати отримані результати та аргументовано захищати прийняті рішення;

- володіти методами та технологіями розробки та оцінювання алгоритмів;

- якісно виконувати роботу та досягати поставленої мети з дотриманням вимог професійної етики.

1 ПЕРЕЛІК ЛАБОРАТОРНИХ РОБІТ

Лабораторна робота № 1

Тема. Програмування лінійних процесів, введення та виведення даних засобами мови C.

Мета: вивчити структуру програм мовою C, вивчити питання алгоритмізації типових обчислювальних процесів, ознайомитися з етапами виконання програм засобами мови C, навчитися складати прості програми лінійних процесів засобами мови C, вивчити формати введення та виведення даних різних типів засобами мови C.

Короткі теоретичні відомості

Кожна програма (якщо вона складна) може бути розбита на декілька простих підпрограм. У мові C/C++ підпрограми називаються *функціями*, а програма на C/C++ являє собою набір із одної або більше функцій.

Будь-яка програма на C/C++ має містити функцію *main*. З неї починається виконання програми.

В кожній функції ліва (*{*) і права (*}*) фігурні дужки позначають початок і кінець виконуючих виразів, ці вирази називаються *тілом функції*.

```
int main()
{
    тіло
}
```

Для написання програм використовуються як синтаксичні (граматичні), так і семантичні (логічні) правила.

Найпростіша програма на мові C/C++ виглядає так:

```
int main( )
{
    return 0;
}
```

Ідентифікатори (*identificators*) використовуються в C/C++ для найменування різних об'єктів. Ідентифікатори складаються із букв (A-Z, a-z), цифр (0-9) і символів

підкреслення (), але при цьому повинні починатися тільки із букв або підкреслення.

Приклад:

Sum_square, J9.

Дані, що необхідні для роботи програми, зберігаються в пам'яті комп'ютера. В C/C++ є декілька типів даних. Спочатку розглянемо цілі типи (цілі числа), а потім – типи з плаваючою комою.

Цілі типи. Типи *char*, *short*, *int* і *long* називаються цілими (integral types). Вони призначені для представлення цілих чисел різної довжини. В такому порядку як записані *char* – представляє найменше число, а *long* – найбільше. Найчастіше використовується тип *int*.

Тип *char* ще використовується для опису даних, що є символами. Символи поміщаються в одинарні лапки: '8' і 8 відрізняються, перший – це символ, а другий – число.

Типи з плаваючою комою (крапкою). Типи з плаваючою крапкою (floating point types), або дійсні, є другою основною різновидністю простих типів C/C++: 18.0; 127.54; 0.57; 4. ; .8.

Типи з плаваючою крапкою мають також різні розміри. У порядку зростання є такі типи з плаваючою крапкою: *float*, *double* і *long double*.

Числа з плаваючою крапкою можуть представлятися і у експоненціальному вигляді: 1.7453E-12; 1.53E11; 7E20.

Останні два типи майже не використовуються, *float* забезпечує необхідну точність.

Ідентифікатори можна використовувати як для іменування констант так і змінних. В C/C++ кожний ідентифікатор може позначати тільки один елемент. Будь-який ідентифікатор перед використанням має бути описаним.

Змінна – це область пам'яті, яка позначена ідентифікатором, в якій зберігаються дані, значення яких змінюються.

Оголосити змінну означає задати як її ім'я, так і її тип. Наступний вираз оголошує змінну *empNum* типу *int*:

```
int empNum;
```

Є можливість оголосити одразу декілька змінних у одному виразі:


```
int studentCount, maxScore;
```

Однак, краще задавати змінні окремими виразами.

Між константами і змінними можна виконувати різні арифметичні операції. В C/C++ існують такі основні арифметичні операції:

“+” – унарний плюс(додавання);

“-” – унарний мінус(віднімання);

“*” – множення;

“/” – ділення з плаваючою крапкою(цілочисельне ділення);

“%” – остача від цілочисельного ділення.($6\%2=0$; $7\%2=1$).

В додаток до арифметичних операцій C/C++ включає операції інкременту (increment) або приросту і декременту (decrement) або від’ємного приросту.

++ – інкремент.

-- – декремент.

Результат приросту полягає у збільшенні цілого або дійсного аргументу на одиницю.

Якщо $num=8$, то $num++=9$

Операції ++ і -- мають дві форми: префіксну ++num; і постфіксну num++; вони виконують одне і те ж, тому вибір запису залежить від програміста.

Рядок *#include* є обов’язковим в програмі і обробляється не компілятором C++, а спеціальною програмою, що називається *препроцесором* (preprocessor). *Препроцесор* – це програма, яка діє як фільтр на етапі компіляції. Перед тим, як потрапити на вхід компілятора, вихідна програма проходить через препроцесор.

Рядок, що починається символом дієзу (#) називається *директивою препроцесора* (preprocessor directive). Препроцесор розширює директиву *#include*, вставляючи вміст вказаного файлу безпосередньо у вихідну програму. Файли, що з’являються в директиві, як правило закінчуються на .h і називаються *файлами заголовків*. Файли заголовків містять оголошення констант, змінних і функцій, необхідних для роботи програми. Кутові дужки < > вказують препроцесору, що файл знаходиться в стандартному каталозі файлів заголовків (include directory).

Між арифметичними операціями існує в C++ такий пріоритет: (), *, /, %, +, -.

У випадку, коли декілька операцій з однаковим пріоритетом слідує одна за

одною, їх порядок групування буде таким же, як і порядок запису цих операцій, тобто зліва на право.

Щоб зробити програми максимально зрозумілими і вільними від помилок використовується явне перетворення типів. Оператор перетворення типів C++ складається із імені типу, після якого в дужках записується перетворюючий вираз:

```
someFloat=float(3*someInt+2);
```

Типи даних можна змішувати у арифметичних виразах:

```
someInt*someFloat, 4.8+someInt-3.
```

Такі вирази є змішаного типу.

Кожний раз, коли ціле значення і значення з плаваючою крапкою є операндами арифметичної операції, відбувається неявне перетворення типів за наступною схемою:

- ціле число тимчасово перетворюється в число з плаваючою крапкою;
- виконується арифметична операція;
- результатом операції стає число з плаваючою крапкою.

Форматувати вивід програми означає керувати його зовнішнім виглядом при відображенні на екрані або принтері.

Хід роботи

Приклад 1.1 Ввести, відлагодити та протестувати лінійну програму згідно умови. Дано: R – радіус основи прямого кругового циліндра; h – висота прямого кругового циліндра. Необхідно: обчислити та вивести на екран: $sb=2*\pi*r*h$ – площу бічної поверхні циліндра; $sp=sb+2*\pi*r*r$ – площу повної поверхні циліндра; $v=\pi*r*r*h$ – об'єм циліндра. Результати розрахунків подати у вигляді таблиці. Реалізувати мовою C.

```
#include <stdio.h>
#include <conio.h>
#include <math.h>

int main() {
    clrscr(); // очищення екрану
    // Оголошення констант та змінних:
    const float pi=3.14159; // оголошення константи pi
    float r; // радіус основи прямого кругового циліндра
    float h; // висота прямого кругового циліндра
    float sb; // площа бічної поверхні циліндра
    float sp; // площа повної поверхні циліндра
    float v; // об'єм циліндра
```

```

// Ввід заданих величин:
printf (" Введіть радіус основи циліндра: ");
scanf ("%f",&r);
printf (" Введіть висоту циліндра: ");
scanf (" %f",&h);

// Обчислення параметрів циліндра:sb=2*pi*r*h;
sp=sb+2*pi*r*r*h;v=pi*r*h*h;
// Виведення результатів у вигляді таблиці:
printf("\n Вхідні величини та результати обчислень:");
printf("\n I -----I");
printf("\n I Вхідні величини: I");
printf("\n I -----I");
printf("\n I Радіус основи циліндра I %8.2f I",r);
printf("\n I -----I");
printf("\n I Висота циліндра I %8.2f I",h);
printf("\n I -----I");
printf("\n I Результати обчислень: I");
printf("\n I -----I");
printf("\n I Бічна поверхня циліндра I %8.2f I",sb);
printf("\n I -----I");
printf("\n I Повна поверхня циліндра I %8.2f I",sp);
printf("\n I -----I");
printf("\n I Об'єм циліндра I %8.2f I",v);
printf("\n I -----I\n");
getch(); //Затримка результатів обчислення на екрані
return 0;
}

```

Результати роботи програми:

Введіть радіус основи циліндра: 4
Введіть висоту циліндра: 5

```

Вхідні величини та результати обчислень:
I -----I
I Вхідні величини: I
I -----I
I Радіус основи циліндра I 4 I
I -----I
I Висота циліндра I 5 I
I -----I
I Результати обчислень: I
I -----I
I Бічна поверхня циліндра I 125.664 I
I -----I
I Повна поверхня циліндра I 628.318 I
I -----I
I Об'єм циліндра I 314.159 I
I -----I

```

Приклад 1.2 Ввести, відлагодити та протестувати програму форматованого виведення даних базових типів мови C.

```

#include <stdio.h>
#include <conio.h>

int main()
{
    clrscr();
    char symv = 'A';
    // Форматований вивід даних цілих типів
    printf(" Виведення десяткового числа 455 форматом %%d :\n");
    printf("  %d\n",455);
    printf(" Виведення десяткового числа 455 форматом %%i :\n");
    printf("  %i\n",455);
    printf(" Виведення десяткового числа +455 форматом %%d :\n");
    printf("  %d\n",+455);
    printf(" Виведення десяткового числа -455 форматом %%d :\n");printf("
    %d\n",-455);
    printf(" Виведення десяткового числа 32000 форматом %%hd :\n");
    printf("  %hd\n",32000);
    printf(" Виведення числа 2000000000 форматом %%ld :\n");
    printf("  %ld\n",2000000000);
    printf(" Виведення вісімкового числа 455 форматом %%o :\n");
    printf("  %o\n",455);
    printf(" Виведення числа -455 форматом %%u :\n");
    printf("  %u\n",-455);
    printf(" Виведення шістнадцяткового числа 455 форматом %%x :\n");
    printf("  %x\n",455);
    printf(" Виведення шістнадцяткового числа 455 форматом %%X :\n");
    printf("  %X\n\n",455);

    // Форматоване виведення даних дійсних типів
    printf(" Виведення дійсного 1234567.89 форматом %%e :\n");
    printf("  %e\n",1234567.89);
    printf(" Виведення дійсного +1234567.89 форматом %%e :\n");
    printf("  %e\n",+1234567.89);
    printf(" Виведення дійсного -1234567.89 форматом %%e :\n");
    printf("  %e\n",-1234567.89);
    printf(" Виведення дійсного -1234567.89 форматом %%e :\n");
    printf("  %e\n",-1234567.89);
    printf(" Виведення дійсного 1234567.89 форматом %%E :\n");
    printf("  %E\n",1234567.89);
    printf(" Виведення дійсного 1234567.89 форматом %%f :\n");
    printf("  %f\n",1234567.89);
    printf(" Виведення дійсного 1234567.89 форматом %%g :\n");
    printf("  %g\n",1234567.89);
    printf(" Виведення дійсного числа 1234567.89 форматом %%G :\n"); //
    printf("  %G\n\n",1234567.89);
}

```

Форматований вивід даних символного типу

```
char str[] = "Це стрічка";
const char *strPtr="Це також стрічка";
printf("\n Виведення символу 'А' за форматом %%c:");
printf("\n %c",symv);
printf("\n Виведення стрічки-константи за форматом %%s:"); printf("\n
%s","Це стрічка");
printf("\n Виведення стрічки з ім'ям str за форматом %%s:");
printf("\n %s",str);
printf("\n Виведення стрічки з допомогою вказівника strPtr за
форматом %%s:");
printf("\n %s",strPtr);
getch();
return 0;
}
```

Результати роботи програми:

```
Виведення десяткового числа 455 форматом %d :
455
Виведення десяткового числа 455 форматом %i :
455
Виведення десяткового числа +455 форматом %d :
455
Виведення десяткового числа -455 форматом %d :
-455
Виведення десяткового числа 32000 форматом %hd :
32000
Виведення числа 1234567890 форматом %ld :
1234567890
Виведення вісімкового числа 455 форматом %o :
707
Виведення числа -455 форматом %u :
65081
Виведення шістнадцяткового числа 455 форматом %x :
1c7
Виведення шістнадцяткового числа 455 форматом %X :
1C7
Виведення дійсного числа 1234567.89 форматом %e :
1.234568e+06
Виведення дійсного числа +1234567.89 форматом %e :
1.234568e+06
Виведення дійсного числа -1234567.89 форматом %e :
-1.234568e+06
Виведення дійсного числа -1234567.89 форматом %e :
-1.234568e+06
Виведення дійсного числа 1234567.89 форматом %E :
1.234568E+06
Виведення дійсного числа 1234567.89 форматом %f :
1234567.890000
Виведення дійсного числа 1234567.89 форматом %g :
1.23457e+06
```

Виведення дійсного числа 1234567.89 форматом %G :
1.23457E+06
Виведення символу 'A' за форматом %c:
A
Виведення стрічки-константи за форматом %s:
Це стрічка
Виведення стрічки з ім'ям str за форматом %s:
Це стрічка
Виведення стрічки з допомогою вказівника strPtr за форматом %s:
Це також стрічка

Приклад 1.3. Ввести, відлагодити та протестувати програму форматowanego виведення даних базових типів даних мови C з використанням форматів d,f,e,E,g,G,p та прапорців '-' '+' '#' '0'.

```
#include <stdio.h>
#include <conio.h>

int main()
{
    clrscr();
    // Виведення додатніх цілих чисел
    // з одночасним вирівнюванням по правому краю:printf("\n%s\n%s\n",
    " Виведення додатніх цілих чисел за форматом %4d",
    " з одночасним вирівнюванням по правому краю:");
    printf(" %4d\n",1);
    printf(" %4d\n",12); printf(" %4d\n",123);
    printf(" %4d\n",1234); printf(" %4d\n",12345);printf("%s\n%s\n",
    " В останньому випадку бракувало ширини формату-", " цим пояснюється
    неякісне вирівнювання");

    printf("\n%s\n%s\n",
    " Виведення від'ємних цілих чисел за форматом %4d", " з одночасним
    вирівнюванням по правому краю:");
    printf(" %4d\n",-1);
    printf(" %4d\n",-12);
    printf(" %4d\n",-123);
    printf(" %4d\n",-1234);
    printf(" %4d\n",-12345);
    printf("%s\n%s\n",
    " В останніх двох випадках бракувало ширини формату -", " цим
    пояснюється неякісне вирівнювання");
    getch();

    clrscr();
    // Використання у форматах прапорця (-)
    // для вирівнювання по лівому краю:
    printf("\n%s\n%s\n",
```

```

" Виведення констант різного типу за форматами з восьми символів",
" з одночасним вирівнюванням по правому краю:");
printf(" %8s%8d%8c%8f\n", "Привіт",7,'a',1.23); printf("\n%s\n%s\n",
" Використання у форматах прапорця (-) ", " для вирівнювання по
лівому краю:");
printf(" %-8s%-8d%-8c%-8f\n", "Привіт",7,'a',1.23);
getch();

clrscr();
// Використання у форматах прапорця (+)
// для обов'язкового Виведення знака чисел:
printf("\n%s\n%s\n",
" Виведення констант цілого типу за форматом %d ",
" з одночасним вирівнюванням по правому краю:");
printf(" %d\n %d\n",786,-786);
printf("\n%s\n%s\n",
" Використання у форматах прапорця (+) ", " для обов'язкового
Виведення знака чисел:");
printf(" %+d\n %+d\n",786,-786);
getch();

clrscr();
// Використання у форматах прапорця (пробіл)
// для Виведення знака пробіл перед додатніми числами:
printf("\n%s\n%s\n",
" Виведення констант цілого типу за форматом %d ",
" з одночасним вирівнюванням по правому краю:");
printf(" %d\n %d\n",786,-786);
printf("\n%s\n%s\n",
" Використання у форматах прапорця (пробіл) ",
" для Виведення знака пробіл перед додатніми числами:");
printf(" %d\n %d\n",786,-786);
getch();

clrscr();
// Використання у форматах прапорця (#) для Виведення:
// знака o - перед вісімковими числами
// знака x - перед шістнадцятковими числами
// крапки - в дійсних числах
int c=1427;
double p=1427.0;
printf("\n%s\n%s\n%s\n%s\n%s\n%s\n",

```

```

" Використання у форматах прапорця (#) ", " для Виведення:",
" знака o - перед вісімковими числами",
" знака x - перед шістнадцятковими числами", " знака X - перед
шістнадцятковими числами", " крапки - в дійсних числах");
printf("\n %#o",c);
printf("\n %#x",c);
printf("\n %#X",c);
printf("\n %g",p);
printf("\n %#g\n",p);
getch();

clrscr();
// Використання у форматах прапорця (0)
// для заповнення порожніх символів формату нулями:
printf("\n%s\n%s\n",
" Використання у форматах прапорця (0) ",
" для заповнення порожніх символів формату нулями:");
printf(" %+09d\n",452);
printf(" %09d\n",452);
getch();
return 0;
}

```

Результати роботи програми:

Виведення додатніх цілих чисел за форматом %4d з одночасним вирівнюванням по правому краю:

```

    1
   12
  123
 1234
12345

```

В останньому випадку бракувало ширини формату - цим пояснюється неякісне вирівнювання

Виведення від'ємних цілих чисел за форматом %4d з одночасним вирівнюванням по правому краю:

```

   -1
  -12
 -123
-1234
-12345

```

В останніх двох випадках бракувало ширини формату - цим пояснюється неякісне вирівнювання

Виведення констант різного типу за форматами з восьми символів одночасним вирівнюванням по правому краю:

з

Привіт 7 a1.230000

Використання у форматах прапорця (-) для вирівнювання по лівому краю:
Привіт 7 a 1.230000

Виведення констант цілого типу за форматом %d з одночасним вирівнюванням по правому краю:

786

-786

Використання у форматах прапорця (+) для обов'язкового Виведення знака чисел:

+786

-786

Виведення констант цілого типу за форматом %d з одночасним вирівнюванням по правому краю:

786

-786

Використання у форматах прапорця (пробіл) для виведення знака пробіл перед додатніми числами:

786

-786

Використання у форматах прапорця (#) для виведення: знака o - перед вісімковими числами знака x - перед шістнадцятковими числами знака X - перед шістнадцятковими числами крапки - в дійсних числах

02623

0x593

0X593

1427

1427.00

Використання у форматах прапорця (0) для заповнення порожніх символів формату нулями:

+00000452

000000452

Завдання на лабораторну роботу

Скласти, відлагодити та протестувати лінійну програму розв'язування задачі , вибраної згідно з варіантом.

Завдання 1.1 Скласти консольний застосунок для обчислення та виведення на екран значень математичних виразів. Значення змінних для підрахунку ввести у діалозі з користувачем.

1.
$$t = \frac{2 \cos \left(\frac{\pi}{6} \right) (1 + z^2)}{\sin(3z)}$$
 При $x=14.26$, $y=-1.22$, $z=3.5 \times 10^{-2}$.

2.
$$u = \frac{\sqrt{3xy^2 + 1} \cdot e^{xz}}{x^2 y^2}$$
 При $x=-4.5$, $y=0.75 \times 10^{-4}$, $z=0.845 \times 10^2$.

3.
$$v = \frac{\ln(xy)}{x - \frac{z}{1 + y^2}}$$
 При $x=3.74 \times 10^{-2}$, $y=-0.825$, $z=0.16 \times 10^2$.

4.
$$w = \frac{\cos \left(\frac{2z^2}{234} \right)}{234}$$
 При $x=0.4 \times 10^4$, $y=-0.875$, $z=-0.475 \times 10^{-3}$.

5.
$$q = \frac{x^2 y^2}{\ln(x^2 + y^2)}$$
 При $x=-15.246$, $y=4.642 \times 10^{-2}$, $z=20.001 \times 10^2$.

6.
$$\beta = \sqrt{\cos(x^2 + y^2)}$$
 При $x=16.55 \times 10^{-3}$, $y=-2.75$, $z=0.15$.

7.
$$\gamma = \frac{1}{\sin(x^2 + y^2)}$$
 При $x=0.1722$, $y=6.33$, $z=3.25 \times 10^{-4}$.

8.
$$\phi = \frac{e^{xy} - 1}{x^2 + y^2}$$
 При $x=-2.235 \times 10^{-2}$, $y=2.23$, $z=15.221$.

9.
$$\psi = \frac{y}{\sqrt[3]{x} \cdot \cos(x^2 + y^2)}$$
 При $x=1.825 \times 10^2$, $y=18.225$, $z=-3.298 \times 10^{-2}$.

10.
$$a = \frac{\sqrt{x^2 + y^2} \cdot e^{xz}}{\sqrt[3]{x^2 + y^2}}$$
 При $x=3.981 \times 10^{-2}$, $y=-1.625 \times 10^3$, $z=0.512$.

11.
$$b = \frac{\sqrt[3]{x^2 + y^2} \cdot \cos(x^2 + y^2)}{e^{xy} + \frac{x}{2}}$$
 При $x=6.251$, $y=0.827$, $z=25.001$.

$$g = \frac{\arctan\left(\frac{\pi}{6}\right)}{x + \frac{1}{y^2 + 1}}$$

12. При $x=3.251$, $y=0.325$, $z=0.466 \times 10^{-4}$.

$$f = \sqrt[4]{y^3 x^2}$$

13. При $x=17.421$, $y=10.365 \times 10^{-3}$, $z=0.828 \times 10^5$.

$$g = \frac{y^4 + 2(x+y)^{3z}}{\sqrt[3]{y^2 z^2 x y}}$$

14. При $x=12.3 \times 10^{-1}$, $y=15.4$, $z=0.252 \times 10^3$.

$$h = \frac{(x+y)^2}{z^2 + 2z}$$

15. За умови, що $x=2.444$, $y=0.869 \times 10^{-2}$, $z=-0.13 \times 10^3$.

Завдання 1.2 Скласти програму для вводу у діалозі значень змінних A, I, C, L, Name та їх відформатованого виводу на екран (значення змінних задано у табл.1.2).

Таблиця 1.2

Варіант	A	I	C	L	Name
1	-14	-10^4	-0,00151	false	Прізвище
2	99,35	72	1995	true	Ім'я
3	0,086	-19	4,025	false	По батькові
4	34	-6124	$3,2 \times 10^{-5}$	true	Прізвище
5	2,008	229	0,019	false	Ім'я
6	$3,5 \times 10^{-4}$	1989	-380,08	true	По батькові
7	0,096	-1	1996	false	Прізвище
8	1,0074	10^2	107,7	true	Ім'я
9	993,285	112000	$2,7 \times 10^{-4}$	false	По батькові
10	-2.1	444	10^3	true	Прізвище
11	3.125	6006006	-13,24	false	Ім'я
12	-45.077	30	$2,5 \times 10^{12}$	true	По батькові
13	12.97	1002	-999,7	false	Прізвище
14	0.09	2004	399,44	true	Ім'я
15	-142	-10^4	-0,00151	false	По батькові
16	9.35	-5072	19,95	true	Прізвище

17	0.86	-19826	4,025	false	Ім'я
18	34	-6	$3,2 \times 10^3$	true	По батькові
19	5.008	-229	-0,019	false	Прізвище
20	$3,5 \times 10^{-4}$	1989	-380,08	true	Ім'я
21	0,095	-12	1996	false	По батькові
22	1,0074	10^2	107,7	true	Прізвище
23	993,285	112000	$2,3 \times 10^{-4}$	false	Ім'я
24	-2.1×10^3	444	10^{-3}	true	По батькові
25	3,125	6007007	-13,24	false	Прізвище
26	,45,07	123	25×10^{12}	true	Ім'я
27	89,09	1000	999,002	false	По батькові
28	-99,78	11	-1,774	true	Прізвище
29	7,99	-30077	1000	false	Ім'я
30	0,124	-100400	-9000	true	По батькові

Зміст звіту

1. Назва та мета роботи.
2. Методика проведення роботи з графічними результатами.

Література: [1–4, 6, 12].

Лабораторна робота № 2

Тема. Програмування лінійних процесів, введення та виведення даних засобами мови C++.

Мета: ознайомлення з концепцією потокового введення та виведення даних різних типів засобами мови C++; ознайомлення з інструментами потокового введення та виведення даних різних типів засобами мови C++; ознайомлення з маніпуляторами потоків мови C++ і особливостями їх застосування у процесах введення виведення даних різних типів.

Короткі теоретичні відомості

В C++ результат обчислень виводиться за допомогою змінної cout (сіаут) і

оператора вставки (<<). *cout*<< „hello”;

Розглянемо, як можна керувати горизонтальними і вертикальними інтервалами при виведенні інформації мовою C++.

Один із способів керування вертикальними інтервалами, це оператор *endl*. Якщо зустрічаються 2-а поспіль оператори *endl*, то у виводі з’являється порожній рядок.

Компілятор визначає кінець виразу за крапкою з комою, а не за фізичним закінченням програмного рядка.

Маніпулятори використовуються тільки у виразах вводу і виводу.

Два інших маніпулятори, *setw* і *setprecision*. *setw* і *setprecision* потребують додатково включити в програму файл заголовків *iomanip.h*.

Маніпулятор *setw* дозволяє керувати кількістю позицій для виводу наступного за маніпулятором елемента даних. *setw* застосовується тільки для форматування чисел і рядків, але не даних типу *char*. Маніпулятор *setprecision*:

```
cout<<setprecision(3)<<x;
```

вказує необхідну кількість знаків після десяткової крапки. Навідміну від *setw*, який діє тільки на елемент, що йде зразу за ним, *setprecision* залишається в силі для всіх наступних елементів вихідного потоку, поки його не буде змінено.

Хід роботи

Приклад 2.1 Ввести, відлагодити та протестувати програму для розв’язування наступної задачі. Дано: R – радіус основи прямого кругового циліндра; h – висота прямого кругового циліндру. Необхідно обчислити та вивести на екран: $sb=2\pi r h$ – площу бічної поверхні циліндру; $sp=sb+2\pi r^2$ – площу повної поверхні циліндру; $v=\pi r^2 h$ – об’єм циліндру. Результати розрахунків подати у вигляді таблиці. Завдання виконати мовою C++.

```
#include <iostream.h>
#include <conio.h>
#include <math.h>

int main()
{
    // Оголошення констант та змінних:
    clrscr(); // очищення екрану
    const float pi=3.14159; // оголошення константи
    float r; // радіус основи прямого кругового циліндра
```

```

float h;    // висота прямого кругового циліндра
float sb;   // площа бічної поверхні циліндра
float sp;   // площа повної поверхні циліндра
float v;    // об'єм циліндра
// Ввід заданих величин:
cout<<" Введіть радіус основи циліндра: ";
cin>>r;
cout<<" Введіть висоту циліндра: ";
cin>>h;
// Обчислення параметрів циліндра:
sb=2*pi*r*h; sp=sb+2*pi*r*r*h;
v=pi*r*h*h;
// Виведення результатів у вигляді таблиці:
cout.precision(3);
cout<<"\n Вхідні величини та результати обчислень:";
cout<<"\n I ----- I";
cout<<"\n I Вхідні величини: I";
cout<<"\n I ----- I";
cout<<"\n I Радіус основи циліндра I "<<r<<" I";
cout<<"\n I ----- I";
cout<<"\n I Висота циліндра I "<<h<<" I";
cout<<"\n I ----- I";
cout<<"\n I Результати обчислень: I";
cout<<"\n I ----- I";
cout<<"\n I Бічна поверхня циліндра I "<<sb<<" I";
cout<<"\n I ----- I";
cout<<"\n I Повна поверхня циліндра I "<<sp<<" I";
cout<<"\n I ----- I";
cout<<"\n I Об'єм циліндра I "<<v<<" I";
cout<<"\n I ----- I\n";
getch();
return 0;
}

```

Результат тестування програми:

Введіть радіус основи циліндра: 4
Введіть висоту циліндра: 5
Вхідні величини та результати обчислень:

```

I ----- I
I Вхідні величини: I
I ----- I
I Радіус основи циліндра I 4 I
I ----- I
I Висота циліндра I 5 I
I ----- I
I Результати обчислень: I
I ----- I
I Бічна поверхня циліндра I 125.664 I
I ----- I
I Повна поверхня циліндра I 628.318 I
I ----- I
I Об'єм циліндра I 314.159 I

```

Ввести, відлагодити та протестувати програму обчислення виразів $a=f1(x,y,z)$ та $b=f2(x,y,z)$ з використанням бібліотеки арифметичних функцій.

$$a = \arccos \frac{y}{2+y^2+|yz|} (\sqrt{\pi+x^2+\sin^2 yz} - \lg \frac{2}{x^2+y^2+1});$$

$$b = e^{-\frac{3x^2+y}{5}-z} + \sqrt[3]{\frac{\lg^2 x}{1+\sin \frac{\pi y}{2}} - \left(\frac{x+y+z}{xyz}\right)^3}.$$

```
#include <iostream.h>
#include <iomanip.h>
#include <windows.h>
#include <conio.h>
#include <math.h>
int main()
{
clrscr();
SetConsoleCP(1251);
SetConsoleOutputCP(1251);
const float pi=3.14159;
float x,y,z;
float a1,a2,a3,a4,a5,a;
float b1,b2,b3,b4,b5,b;
// Ввід заданих величин:
cout<<" Введіть значення трьох дійсних змінних:\n"; cin>>x>>y>>z;

// Обчислення виразу a:
a1=y/(2+y*y+fabs(y*z));
a2=acos(a1);
a3=pi+x*x+pow(sin(y*z),2);
a4=pow(a3,1/2);
a5=log10(2/(x*x+y*y+1));
a=a2*(a4-a5);

// Обчислення виразу b:
b1=exp(-3*x*x+y/3-z);
b2=pow(tan(x),2);
b3=b2/(1+sin(pi*y/2));
b4=pow(b3,1/3);
b5=(x+y+z)/(x*y*z);
b=b1+b4-pow(b5,2);

// Виведення результатів у вигляді таблиці:
cout<<fixed<<setprecision(3)<<"\n Вхідні величини та результати
обчислень:";
cout<<"\n I .....I";
cout<<"\n I Вхідні величини: I";
cout<<"\n I .....I";
cout<<"\n I x= I"<<" "<<x<<" I";
cout<<"\n I .....I";
cout<<"\n I y= I"<<" "<<y<<" I";
```

```

cout<<"\n I -----I";
cout<<"\n I          z=          I"<<"          "<<z<<" I";
cout<<"\n I -----I";
cout<<"\n I          Результати обчислень:          I";
cout<<"\n I -----I";
cout<<"\n I a1= I          "<<a1<<" I          b1= "<<b1<<"I";
cout<<"\n I -----I";
cout<<"\n I a2= I          "<<a2<<" I          b2= "<<b2<<"I";
cout<<"\n I -----I";
cout<<"\n I a3= I          "<<a3<<" I          b3= "<<b3<<"I";
cout<<"\n I -----I";
cout<<"\n I a4= I          "<<a4<<" I          b4= "<<b4<<"I";
cout<<"\n I -----I";
cout<<"\n I a5= I          "<<a5<<" I          b5= "<<b5<<" I";
cout<<"\n I -----I";
cout<<"\n I a = I          "<<a<<" I          b = "<<b<<" I";
cout<<"\n I -----I\n";
getch();
return 0;
}

```

Результати тестування програми:

Введіть значення трьох дійсних змінних:1 2 3

Вхідні величини та результати обчислень:

```

I -----I
I          Вхідні величини:          I
I -----I
I          x=          I          1.00          I
I -----I
I          y=          I          2.00          I
I -----I
I          z=          I          3.00          I
I -----I
I          Результати обчислень:          I
I -----I
I a1= I          0.17 I          b1= I 0.00 I
I -----I
I a2= I          1.40 I          b2= I 2.43 I
I -----I
I a3= I          4.22 I          b3= I 2.43 I
I -----I
I a4= I          1.00 I          b4= I 1.00 I
I -----I
I a5= I          -0.48 I          b5= I 1.00 I
I -----I
I a = I          2.07 I          b = I 0.00 I
I -----I

```

Ввести, відлагодити та протестувати програму потокового виведення даних базових типів мови C++ з використанням відповідних маніпуляторів.

```
#include <iostream.h>
```



```

#include <iomanip.h>
#include <conio.h>

char ac='A', bc='B', cc, dc;
int ai=18, bi=-18, ci, di;
unsigned int au=18, bu=24, cu, du;
float af=12.34567,bf=12.234567,cf,df;
int main () {
clrscr();

//Введення/виведення даних символьного типу
cout<<"\n Введіть значення змінних cc та dc символьного типу:"
<<endl; cin>>cc>>dc;
cout<<"\n Значення змінної ac="<<setw(10)<<right<<ac<<endl; cout<<"\n
Значення змінної bc="<<setw(10)<<left<<bc<<endl; cout<<"\n Значення
змінної cc="<<setw(10)<<right<<cc<<endl; cout<<"\n Значення змінної
dc=";
cout.width(10);
cout<<left<<dc<<endl<<endl;

// Введення/виведення даних цілого типу зі знаком
cout<<"\n Введіть значення змінних ci та di цілого типу зі знаком:
"<<endl;
cin>>ci>>di;
cout<<"\n Значення змінної ai="<<setw(10)<<right<<ai<<endl; cout<<"\n
Значення змінної bi="<<setw(10)<<right<<bi<<endl; cout<<"\n Значення
змінної ci="<<setw(10)<<left<<ci<<endl; cout<<"\n Значення змінної
di="<<setw(10)<<left<<di<<endl<<endl;

// Введення/виведення даних цілого типу без знаку
cout<<"\n Введіть значення змінних cu та du цілого типу без знаку:
"<<endl;
cin>>cu>>du;
cout<<"\n Значення змінної au=\n"
<<setw(5)<<dec<<right<<au<<" - в 10-й нотації \n"
<<setw(5)<<oct<<right<<au<<" - у 8-й нотації \n"
<<setw(5)<<hex<<right<<au<<" - в 16-й нотації "<<endl<<endl; cout<<"\n
Значення змінної bu=\n"
<<setw(5)<<dec<<left<<bu<<" - в 10-й нотації \n"
<<setw(5)<<oct<<left<<bu<<" - у 8-й нотації \n"
<<setw(5)<<hex<<left<<bu<<" - в 16-й нотації n"<<endl<<endl; cout<<"\n
Значення змінної cu=\n"
<<setw(5)<<dec<<right<<cu<<" - в 10-й нотації \n"
<<setw(5)<<oct<<right<<cu<<" - у 8-й нотації \n"
<<setw(5)<<hex<<right<<cu<<" - в 16-й нотації "<<endl<<endl; cout<<"\n
Значення змінної du=\n"
<<setw(5)<<dec<<left<<du<<" - в 10-й нотації \n"
<<setw(5)<<oct<<left<<du<<" - у 8-й нотації \n"
<<setw(5)<<hex<<left<<du<<" - в 16-й нотації "<<endl<<endl;

// Введення/виведення даних дійсного типу
cout<<"\n Введіть значення змінних cf та df дійсного типу: "<<endl;
cin>>cf>>df;
cout<<setprecision(4);

```

```

cout<<"\n Значення змінної af="<<right<<fixed<<af<<endl; cout<<"\n
Значення змінної bf="<<right<<fixed<<bf<<endl; cout<<"\n Значення
змінної cf="<<left<<scientific<<cf<<endl; cout<<"\n Значення змінної
df="<<left<<scientific<<df<<endl;
cout<<"\n Значення змінної af="<<setw(12)<<showpoint<<
setfill('X')<<right<<fixed<<af<<endl;
cout<<"\n Значення змінної af="<<setw(12)<<showpoint<<
setfill('X')<<left<<fixed<<af<<endl;
cout<<"\n Значення змінної af="<<setw(12)<<showpoint
<<setfill('X')<<right<<scientific <<af<<endl;
cout<<"\n Значення змінної af= " << setw(12) << showpoint <<
setfill('X') << left << scientific << af << endl;
getch();
return 0;
}

```

Результати тестування програми:

Введіть значення змінних ss та dc символного типу:
C D

Значення змінної ac= A
Значення змінної bc= B
Значення змінної cc= C
Значення змінної dc= D

Введіть значення змінних ci та di цілого типу зі знаком:
123 -123
Значення змінної ai= 18
Значення змінної bi= -18
Значення змінної ci= 123
Значення змінної di= -123

Введіть значення змінних su та цілого типу без знаку:
124 421

Значення змінної au=
18 - in 10-й нотації
22 - in 8- й нотації
12 - in 16- й нотації

Значення змінної bu=
24 - in 10- й нотації
30 - in 8- й нотації
18 - in 16- й нотації

Значення змінної su=
124 - in 10- й нотації
174 - in 8- й нотації
7c - in 16- й нотації

Значення змінної du=
421 - in 10- й нотації
645 - in 8- й нотації
1a5 - in 16- й нотації

Введіть значення змінних cf та df дійсного типу:
35.3456789 -35.3456789

Значення змінної af=12.3457
Значення змінної bf=12.2346
Значення змінної cf=3.5346e+01
Значення змінної df=-3.5346e+01

Значення змінної af=XXXXX12.3457
Значення змінної af=12.3457XXXXX
Значення змінної af=XX1.2346e+01
Значення змінної af=1.2346e+01XX

Завдання на лабораторну роботу

Скласти, відлагодити та протестувати лінійну програму розв'язування задачі , вибраної згідно з варіантом.

Завдання 2.1 Скласти програму мовою C++ для вводу у діалозі значень змінних A, I, C, L, Name та їх відформатованого виводу на екран (значення змінних задано у табл.1.2).

Завдання 2.2 Підрахувати значення функції F відповідно до варіанту. При отриманні в знаменнику значення 0 вивести на екран повідомлення.

$$1) \quad F = \frac{\min(x, y) + 0,5}{(\max(x, y))^2 - \sin z}$$

$$2) \quad F = \frac{\min(x, y, z) + x}{(\max(x, z))^2 + y}$$

$$3) \quad F = \frac{\max(x, y) + y}{(\min(x, y, z))^2 + yx}$$

$$4) \quad F = \frac{\min(z, \max(x, y))}{x^2 + z}$$

$$5) \quad F = \frac{\max(x^2, y^2, xz) + x}{(\min(x, y))^2 - y}$$

$$6) \quad F = \frac{\min(x, y + z)}{\max(x^2, y) + z^3}$$

$$7) \quad F = \frac{\max(x^2, y^2, x - y) + x}{(\min(x, y))^2 + y^4}$$

$$8) \quad F = \frac{\min(x, (x + y)^2)}{x^2 + \max(y^3, x)}$$

$$9) \quad F = \frac{\max(x + z, \min(x, y))}{x^2}$$

$$10) \quad F = \frac{\min(x, \max(x + y, z))^2}{x^2 + z^2}$$

$$11) \quad F = \frac{\min(x, y + z)}{\max(x, y) + \sin z}$$

$$12) \quad F = \frac{\min(x, y - x)}{\max(yz, x^2) + \cos 2z^3}$$

$$13) F = \frac{\max(x^2, z^2) + \cos 2x^2}{(\min(x, y))^2 - y}$$

$$14) F = \frac{\min(x^2, y+z)}{x^2 + \max(z^3, xy)}$$

$$15) F = \frac{\max(x+y, \max(x, zy))}{xe^2}$$

$$16) F = \frac{\max(x^3, y^2, xy) + x}{(\min(x, yz))^2 - y}$$

$$17) F = \frac{x(\max(x+z, zy))}{\min(x, y) + x^2} 1$$

$$18) F = \frac{\min(x, \max(x+z, y))^2}{x^3 + z^2}$$

$$19) F = \frac{x^3 + \max(z^2, y)}{(\max(x, z))^2 - y}$$

$$20) F = \frac{\max(x, y+z) + e^{xz}}{\min(x^2, y) + z^3}$$

$$21) F = \frac{\min(x^2, z^4, xy) + x}{(\max(x, y))^2 - y}$$

$$22) F = \frac{\min(x, y+z) + e^x}{\max(x^2, y) + z^3}$$

$$23) F = \frac{\max(x^2, y^2) + \cos 4z^2}{\min(x, y) + x^2}$$

$$24) F = \frac{(\min(x, y))^2 - y}{x^2 + \max(z^3, x)}$$

$$25) F = \frac{\min(x, y+2x)}{\max(y, z) + \sqrt[3]{x}}$$

$$26) F = \frac{\min(x, y-z)}{\max(yz, x^2) + \cos 2x^2}$$

$$27) F = \frac{\max(x^3, z^2) + \cos 4y^2}{\min(y, yz) + \sqrt{x}}$$

$$28) F = \frac{(\min(x, y))^4 + 2e^x}{x^3 + \max(z^2, x)}$$

$$29) F = \frac{\max(xz, \min(y, z))}{x^2 + \sin zy}$$

$$30) F = \frac{\max(x, \max(y, z))^4}{\sin 2y + xe^2}$$

Зміст звіту

1. Назва та мета роботи.
2. Методика проведення роботи з графічними результатами.

Література: [1–6, 8, 10].

Лабораторна робота № 3

Тема. Функції користувача з параметрами- значеннями засобами мови C/C++.

Мета: ознайомлення з технологією створення функцій користувача з параметрами.

Короткі теоретичні відомості

Будь-який програмний комплекс C++ містить стандартну бібліотеку функцій, що виконують стандартні обчислення.

Використовувати бібліотечну функцію не важко. Перш за все треба помістити на початку програми директиву `#include` з означенням відповідного файлу заголовків. Приклади бібліотечних функцій:

Файл заголовків	Функція	Тип параметрів	Тип результату	Результат
<code><stdlib.h></code>	<code>abs(i)</code>	int	int	Модуль i
<code><math.h ></code>	<code>cos (x)</code>	float	float	Косинус x (x в радіанах)
<code><math.h ></code>	<code>fabs (x)</code>	float	float	Модуль числа x
<code><stdlib.h ></code>	<code>fabs(x)</code>	long	long	Модуль числа x
<code><math.h ></code>	<code>pow(x, y)</code>	float	float	Піднесення x в степінь y (якщо $x=0.0$, то $y>0$, якщо $x\neq 0.0$, то y має бути цілим числом)
<code><math.h ></code>	<code>sin(x)</code>	float	float	Синус x (x в радіанах)
<code><math.h ></code>	<code>sqrt(x)</code>	float	float	Квадратний корінь із x ($x>0.0$)

Стандартна бібліотека C++ містить десятки функцій.

Для структурування програми зазвичай використовують функції, написані користувачем, що полегшує роботу з програмним кодом, його документуванням та розумінням іншими розробниками. У наступних прикладах показано створення і використання функцій користувача.

Хід роботи

Приклад 3.1. Ввести, відлагодити та протестувати програму для розв'язування наступної задачі. Дано: R – радіус основи прямого кругового циліндра; h – висота прямого кругового циліндра. Необхідно обчислити та вивести на екран: $sb=2*\pi*r*h$ – площу бічної поверхні циліндра; $sp=sb+2*\pi*r*r$ – площу повної поверхні циліндра; $v=\pi*r*r*h$ – об'єм циліндра. Завдання виконати мовою C з використанням функцій користувача з параметрами.

```
#include <stdio.h>
#include <conio.h>
#include <math.h>
#define pi 3.14159

// Прототипи функцій:
float BichPoverchnja(float rc, float hc);
float PovPoverchnja(float rc, float hc);
float Objem(float rc, float hc);

int main()
{
    clrscr();
    // Оголошення констант та змінних:
    float r; // радіус основи прямого кругового циліндра
    float h; // висота прямого кругового циліндра
    float sb; // площа бічної поверхні циліндра
    float sp; // площа повної поверхні циліндра
    float v; // об'єм циліндра
    // Ввід заданих величин радіуса та висоти:
    printf ("\n Введіть радіус основи циліндра: ");
    scanf("%f",&r);
    printf ("\n Введіть висоту циліндра: ");
    scanf("%f",&h);
    // Обчислення параметрів циліндра:
    printf ("\n Площа бічної поверхні циліндра sb=%f\n",
    BichPoverchnja(r,h));
    printf (" Площа повної поверхні циліндра sp=%f\n",
    PovPoverchnja(r,h));
    printf (" Об'єм циліндра    v=%f\n", Objem(r,h));
    getch();
    return 0;
}

// Підпрограма-функція обчислення площі бічної поверхні
float BichPoverchnja(float rc, float hc)
{
    return 2*pi*rc*hc;
};

// Підпрограма-функція обчислення площі повної поверхні
float PovPoverchnja(float rc, float hc)
{
    return BitshPoverchnja(rc,hc)+2*pi*rc*rc;
```

```

}
// Підпрограма-функція обчислення об'єму циліндра
float Objem(float rc, float hc)
{
    return pi*rc*rc*hc;
}

```

Результати роботи програми:

Введіть радіус основи циліндра: 4
Введіть висоту циліндра: 9

Площа бічної поверхні циліндра sb=226.194480
Площа повної поверхні циліндра sp=326.725360
Об'єм циліндра v=452.388960

Завдання на лабораторну роботу

Скласти, відлагодити та протестувати лінійну програму обчислення значень змінних *a* та *b* при заданих значеннях аргументів *x*, *y* та *z* відповідно до завдання 2.2 попередньої лабораторної роботи, склавши для обчислення допоміжних величин *a1*, *a2*, і т.д. та *b1*, *b2*, і т.д. за необхідності, функції з параметрами.

Зміст звіту

1. Назва та мета роботи.
2. Методика проведення роботи з графічними результатами.

Література: [1, 3, 6, 9].

Лабораторна робота № 4

Тема. Програмування розгалужених процесів за допомогою оператора **if**.

Мета: вивчення оператор **if** мови C/C++, вивчення команди **?** мови C/C++, ознайомлення з технологією створення розгалужених програм засобами мови C/C++.

Короткі теоретичні відомості

Порядок, у якому вирази виконуються в програмі, називається *поток* керування (flow of control). Потік керування як правило є послідовним. У місцях, де потрібно змінити напрям потоку керування, використовуються *керуючі структури*

(control structures). Це спеціальні оператори, які передають керування виразу, що не є наступним за порядком. Керуюча структура вибору використовується для вибору між взаємовиключаючими діями. Для цього формулюється деяке твердження, яке або істинне, або ні. Якщо воно істинне, то комп'ютер виконує один вираз, якщо ні, то інший. Оператор *if*, або умовний (relational) оператор, дозволяє маніпулювати потоком керування. В C++ умовний оператор має дві форми: *if-then-else* і *if-then*. Розглянемо його представлення у вигляді *if-then-else*.

Синтаксис:

if (деяка умова істинна)

then виконати певну дію;

else виконати іншу дію;

then можна опускати для полегшення читання коду.

Приклад:

```
if (hours <= 40.0) pay = rate * hours;
```

```
else pay = rate * (40.0 + (hours - 40.0) * 1.5);
```

```
cout << pay;
```

Помістивши послідовність операторів у фігурні дужки {}, ми перетворюємо ці оператори в єдиний вираз:

```
if (divisor != 0)
```

```
{
```

```
    result = dividend / divisor;
```

```
    cout << "Істина" << endl;
```

```
}
```

```
else
```

```
{
```

```
    cout << "Помилка" << endl;
```

```
    result = 9999;
```

```
}
```

Інколи можна стикнутися з ситуацією, яка описується так: „якщо деяка умова істинна, необхідно виконати певну дію, в іншому випадку ніяких дій застосовувати не потрібно”.

```
if (a <= b)
```

```
    c = 20;
```

```
else
```

```
    ;
```


А краще зовсім опустити else, в результаті отримаємо умовний оператор у формі *if then*.

```
if (age<18) cout<<"Не може бути виборцем";  
cout<<"Виборець"<<endl;
```

Умовний оператор *if-then* так як і *if-then-else* може містити блок. Наприклад: „Відніміть значення у рядку 23 від значення у рядку 17 і запишіть результат у рядок 24; якщо результат від’ємний, запишіть нуль і відмітьте пункт 24А”.

```
result=line17-line23  
if (result<0.0)  
{  
    cout<<"пункт 24А"<<endl;  
    result= 0.0;  
}  
line24=result;
```

В C++ не існує обмежень на тип виразів, що входять в умовний оператор. Отже, всередині одного оператора *if* може знаходитись інший оператор *if*. При розміщенні *if* всередині *if* створюється так звана вкладена керуюча структура (*nested control structure*).

Коли умовні оператори вкладені, легко заплутатися до якого *if* належить те чи інше *else*. Існує просте правило якого дотримується компілятор C++, при відсутності фігурних дужок, *else* завжди відноситься до найближчого *if*, що немає парного *else*. Нам хотілося б, щоб *else* відносилось до зовнішнього, а не до внутрішнього *if*, але це не так, бо форматування не впливає на приналежність.

```
if(average>=60.0) if (average<70.0)  
cout<<"Добре";  
else cout<<"Не здано";
```

Треба записати так:

```
if (average>=60.0)  
{  
    if (average<70.0) cout<<"Добре здано";  
}  
else  
    cout<<"Не здано";
```

Фігурні дужки вказують, що внутрішній умовний оператор закінчений, тому *else* ставиться у відповідність зовнішньому *if*.

C++ забезпечує спосіб перевірки, чи знаходиться потік у стані відмови, чи ні. Для цього потокова змінна просто підставляється у логічний вираз, так ніби це є булева змінна:

```
if (cin)
... if (! inFile)
```

...

Це називається перевіркою стану потоку (testing the state of a stream). Результатом такої перевірки є або ненульове значення, що означає, що остання операція вводу/виводу з даним потоком була успішною, або нуль, що відповідає помилці вводу/виводу.

Треба пам'ятати, що потік, перейшовши у стан відмови, в ньому і залишається.

Покажемо, як перевірити, чи був вхідний файл успішно відкритий.

```
#include <iostream.h>
#include <fstream.h> // Для файлового вводу-виводу
int main()
{
    int height;
    int width;
    ifstream inFile;
    inFile.open("mydata.dat"); // Спроба відкрити файл для читання
    if (!inFile) // Чи відкритий файл?
    {
        cout<<"Файл не відкритий"; // Вивід повідомлення
        return 1; // і завершаємо програму
    }
    inFile>>height>>width; // Зчитуємо дані
    .
    .
    .
    return 0;
}
```

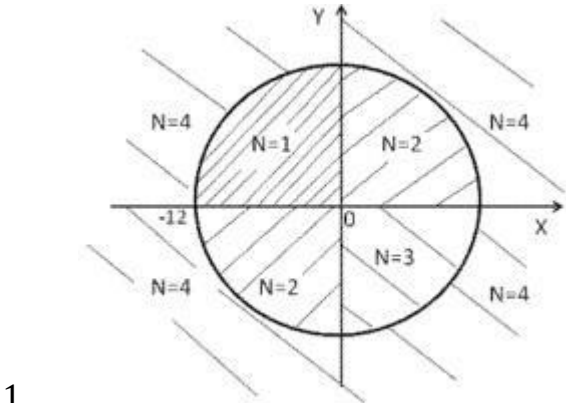
Кожен раз, коли відкривається файл даних для читання, треба перевіряти стан потоку до того, як приступити до роботи з цим файлом.

Хід роботи

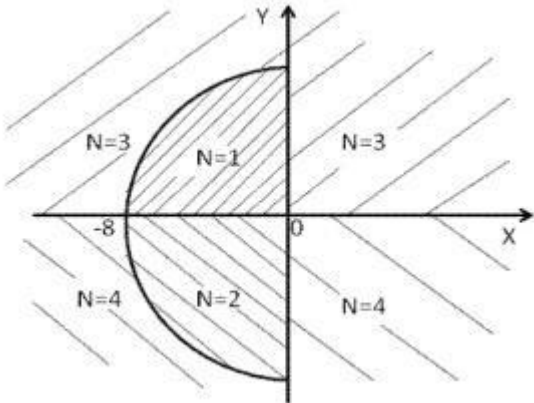
Ввести, відлагодити та протестувати програму для розв'язування такої задачі. Дано координати точки x та y точки $M(x,y)$ координатної площини. Визначити, чи

належить ця точка області, заштрихованій на рисунку відповідно до заданого варіанту завдання. Межі областей відносити до області з найбільшим номером

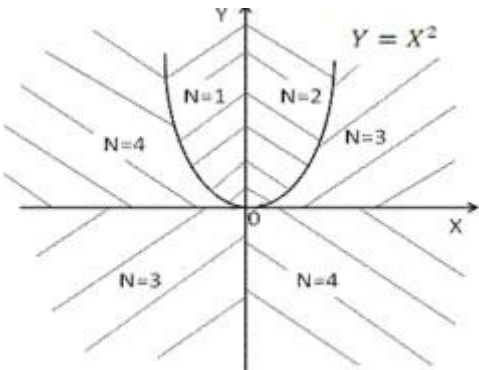
Індивідуальні завдання



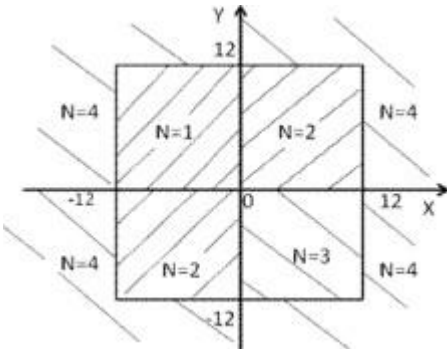
1.



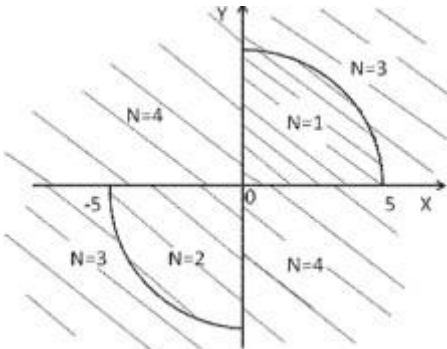
2.



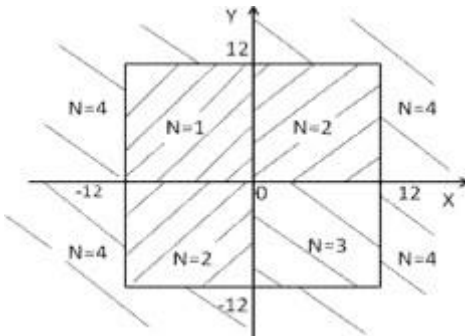
3.



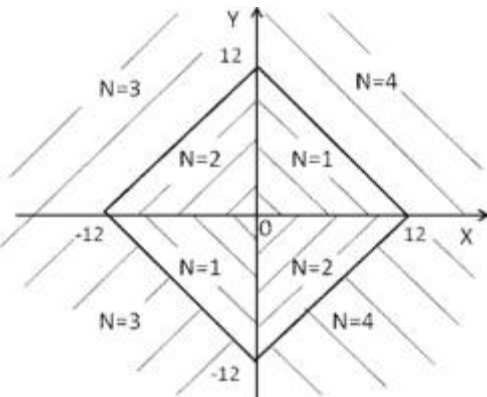
4.



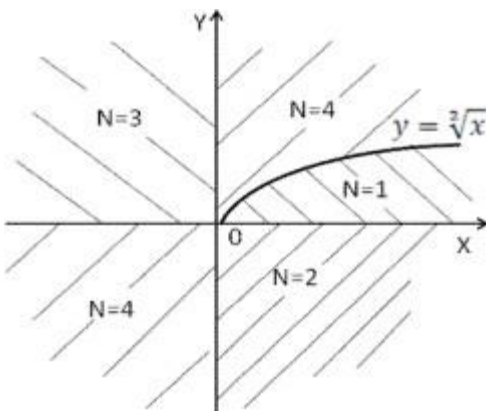
5.



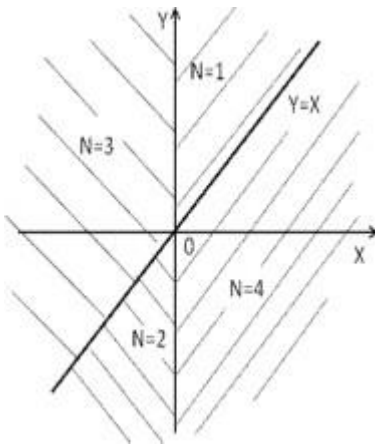
6.



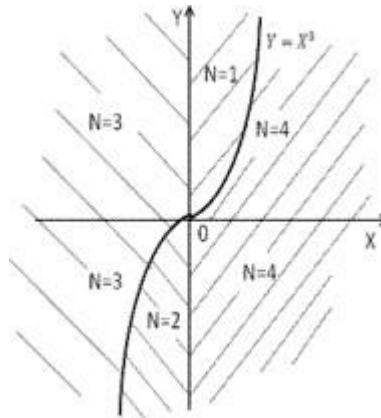
7.



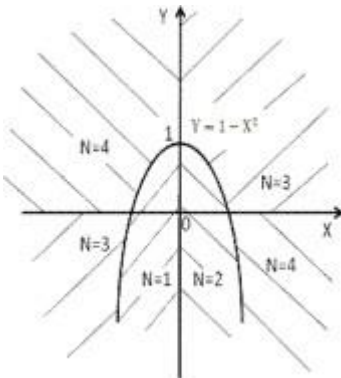
8.



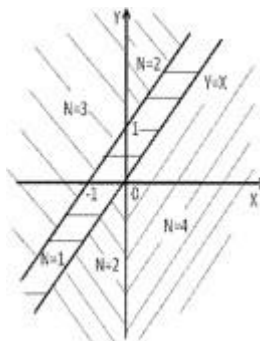
9.



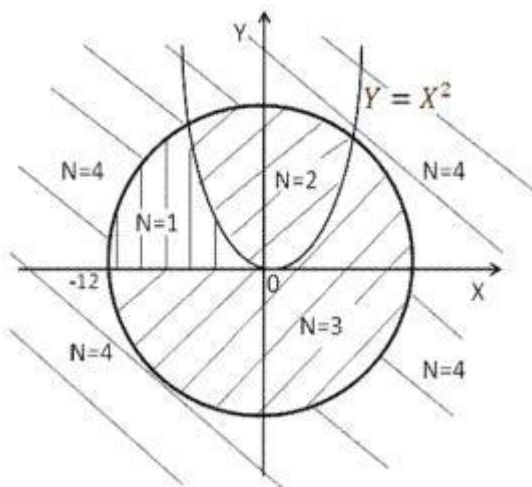
10.



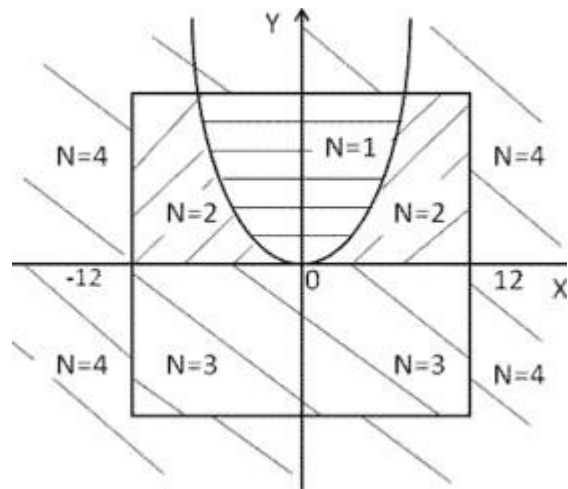
11.



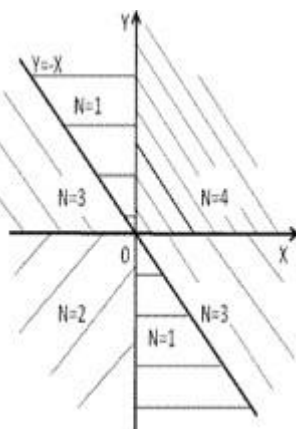
12.



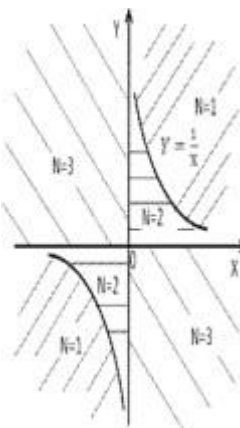
13.



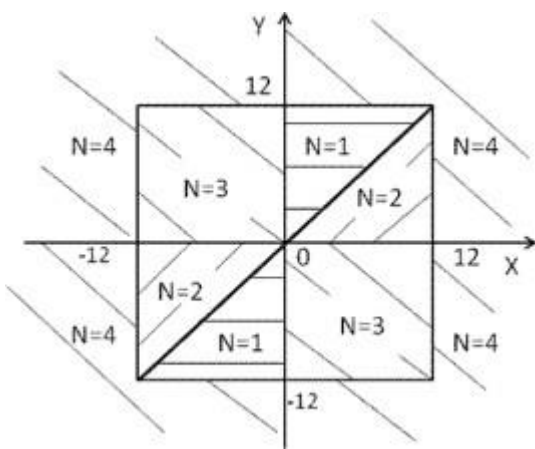
14.



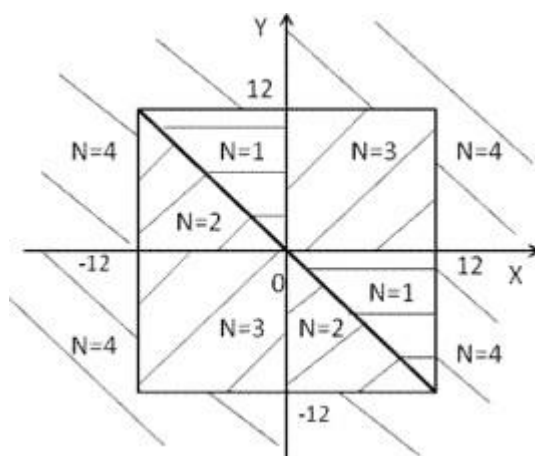
15.



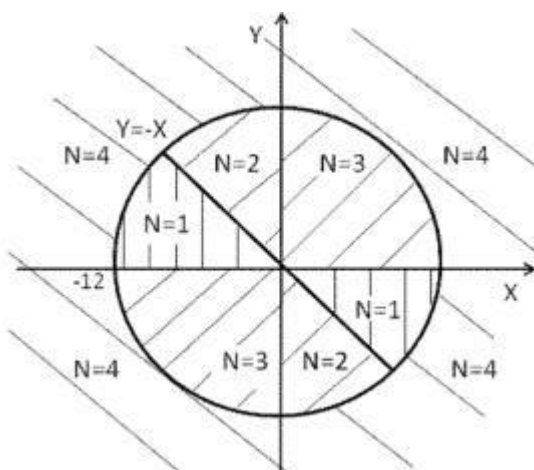
16.



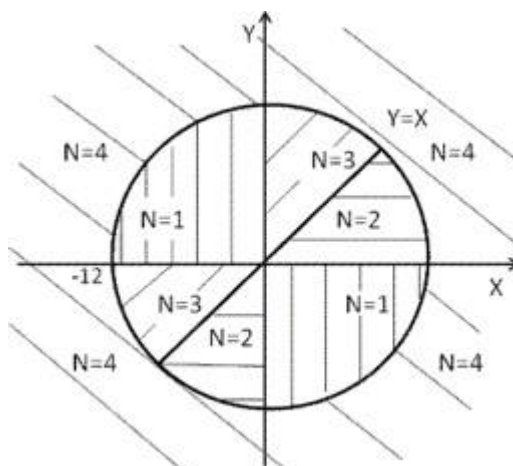
17.



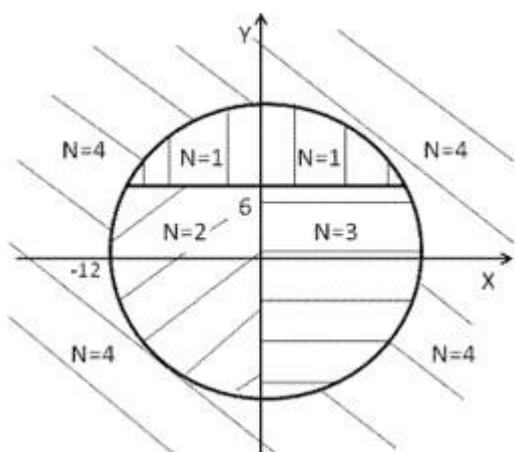
18.



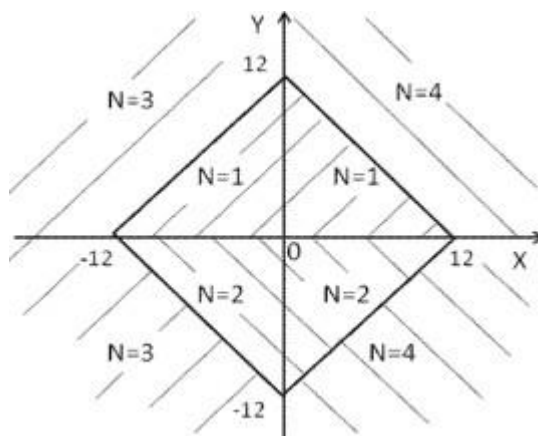
19.



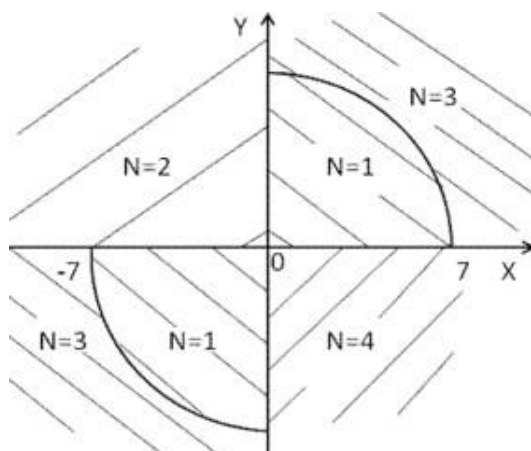
20.



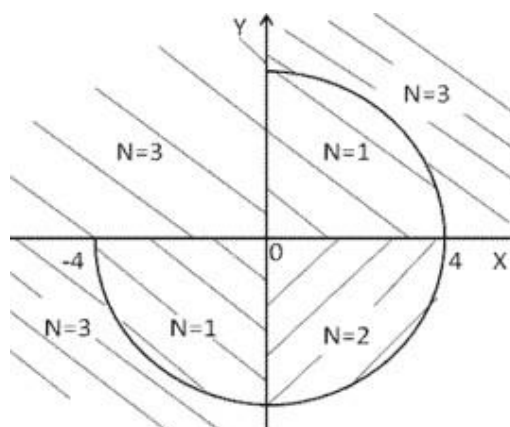
21.



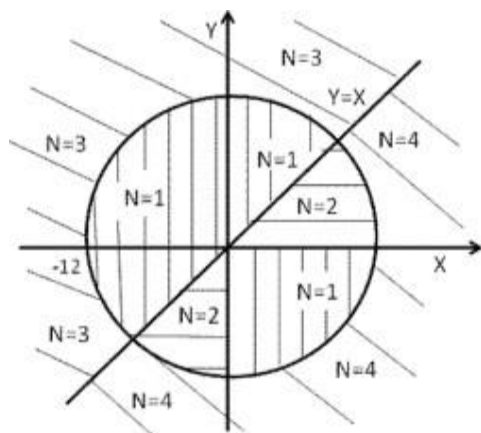
22.



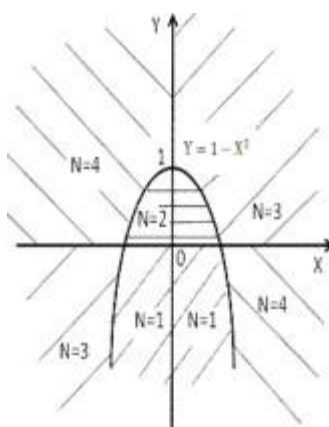
23.



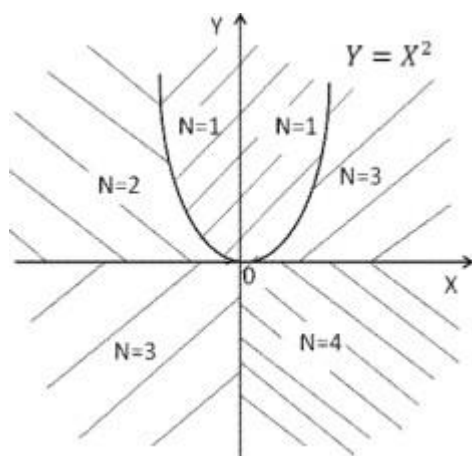
24.



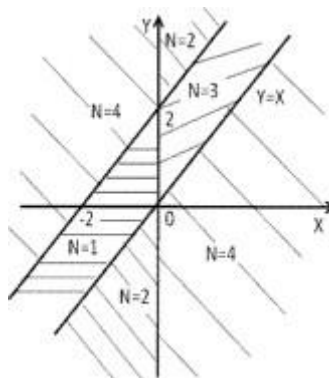
25.



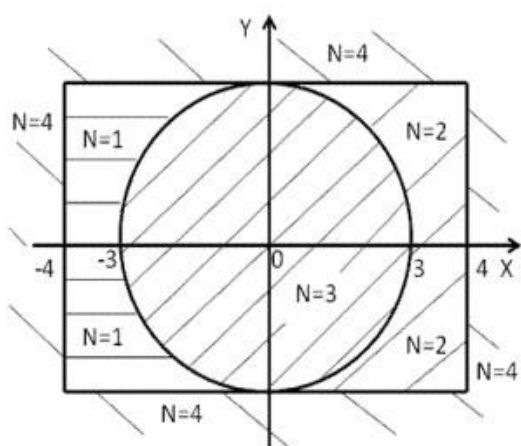
26.



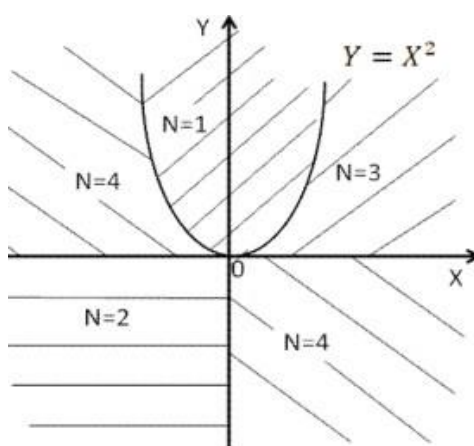
27.



28.



29.



30.

Зміст звіту

1. Назва та мета роботи.
2. Методика проведення роботи зі скрін-шотами результатів виконання програми.

Література: [1–4, 6, 12].

Лабораторна робота № 5

Тема. Програмування циклічних процесів мовою C/C++

Мета: вивчення операторів циклу мови C/C++.

Короткі теоретичні відомості

Цикл while.

Оператор while, як оператор if, перевіряє виконання деякої умови.

Приклад.

```
while(inputVal !=25) cin>>inputVal;
```

Вираз while являє собою циклічну керуючу структуру. Оператор який повторюється кожний раз, коли виконується той чи інший цикл, називається тілом (body). У прикладі тіло циклу – це оператор, що зчитує значення inputVal. Конструкція while має наступний зміст: "Якщо значення умови відмінне від нуля (true), то виконати тіло циклу".

Тіло циклу виконується за декілька кроків;

- момент, коли потік керування передається першому оператору всередині тіла циклу, називається *точкою входу* (loop entry) даного циклу.

наступний прохід через цикл здійснюється кожний раз, коли виконується його тіло; такий прохід називається *ітерацією* (iteration);

- перед кожною ітерацією керування передається на *перевірку умови* (loop test) на початку циклу;

- коли остання ітерація завершена і керування перейшло до оператора, наступному безпосередньо після циклу, говорять, що програма вийшла із циклу.

Умова, що приводить до закінчення циклу, називається умовою *завершення циклу* (termination condition).

Завершення циклу while відбувається при рівності нулю (false) умови в точці його перевірки.

Основні типи циклічних процесів: цикли, що керуються лічильником (countcontrolled), які повторюються вказане число раз, і цикли, що керуються подією (eventcontrolled), які повторюються до тих пір, поки в середині циклу не відбудеться певна подія.

Цикл, що керується лічильником при аналізі умови використовує змінну циклу. Перед входом у цикл, що керується лічильником, необхідно ініціалізувати змінну керування циклу, тобто присвоїти їй певне початкове значення, а потім це значення протестувати. Потім при кожній наступній ітерації змінна циклу отримує приріст.

```
loopCount=1;           // Ініціалізація
While (loopCount<=10) // Перевірка умови.
{
    ...                // Повторюючи дії.
    loopCount++;       // Приріст лічильника.
}
```

Якщо програма виконується набагато довше очікуваного часу, то є імовірність, що в програмі присутній нескінченний цикл.

Цикл do-while.

Вираз *do-while* – це циклічна керуюча структура, в якій умова перевіряється у кінці циклу. Це гарантує виконання тіла циклу принаймні один раз.

```
do
{
    вираз 1; вираз 2;
    .
    .
    .
    вираз n;
}
while(умова);
```

Тут виконуються вирази між словами *do* і *while* до тих пір, поки “умова” є відмінною від нуля (*true*) в кінці циклу.

Порівняємо цикли *while* і *do-while*, які знаходять першу крапку у вхідному файлі даних:

<pre>dataFile>>inputChar; while (inputChar!='.') dataFile>inputChar;</pre>	<pre>do dataFile>>inputChar; while (inputChar!='.');</pre>
---	--

У першому випадку треба здійснити першочергове читання, щоб присвоїти змінній *inputChar* значення перед входом у цикл. Цього не потрібно робити у другому випадку, оскільки вираз вводу в тілі циклу виконується перед перевіркою умови циклу. Наведемо приклад порівняння віку:

```
cout << "введіть вік: ";
cin>>age;
while (age<=0)
```



```

{
    cout<<"вік має бути позитивною величиною."<<endl;
    cout<<"введіть вік:";
    cin>>age;
}

```

А тепер за допомогою циклу do-while:

```

do
{
    cout<<"введіть вік:";
    cin>>age;
    if (age<=0)
        cout<<"вік має бути позитивною величиною."<<endl;
}
while (age<=0);

```

Бачимо, що у другому випадку не треба двічі вставляти в текст вивід запрошення, але в ньому перевірка здійснюється двічі.

Цикл *do-while* можна використовувати для реалізації циклів, що керуються лічильником, якщо попередньо відомо, що тіло циклу завжди виконується принаймні один раз. Наведемо приклад обчислення суми цілих чисел від 1 до *n*.

<pre> sum=0; counter=1; while (counter<=n) { sum=sum+counter; counter++; } </pre>	<pre> sum=0; counter=1; do { sum=sum+counter; counter++; } </pre>
--	---

Якщо *n* – позитивне число, то ці версії еквівалентні. Але якщо *n* дорівнює 0 або від’ємне, то два цикли дають різні результати. У першому випадку *sum*=0, оскільки тіло циклу не виконається жодного разу, а в другому випадку значення суми буде відмінне від нуля, оскільки тіло циклу раз виконується, після чого виконується перевірка умови.

Цикл *while* називається циклом *попередньої перевірки*. Цикл *do-while* називається циклом *постперевірки*.

Цикл for.

Вираз *for* полегшує написання циклів, що керуються лічильником. Наступний фрагмент виводить цілі числа від 1 до *n*:

```

for (count=1; count<=n; count++)
    cout<<count<<endl;

```

for – це більш коротка форма циклу *while*. Фактично, компілятор перетворює вираз *for* у еквівалентний цикл *while*.

Цикли *for* можуть бути так само вкладеними, як і *while* чи *do-while*.

```
for (lastNum=1; lastNum<=7; lastNum++)
{
    for (numToPrint=1; numToPrint<=lastNum; numToPrint++)
        cout<<numToPrint;
    cout<<endl;
}
```

Вивід буде таким:

```
1
12
123
1234
12345
123456
1234567
```

Хід роботи

Ввести, відлагодити та протестувати програму для розв'язування задачі згідно індивідуального варіанту.

Завдання на лабораторну роботу

Завдання 5.1 Побудувати таблицю відповідностей між мірами. Початкове значення міри, крок зміни цього значення та кількість рядків у таблиці (10-15) задати самостійно у режимі діалогу. Оформити таблицю якнайкраще, використовуючи формати виведення.

1. 1 унція=28.353495 г=142 карати;
2. 1 драхм=1.77185 г=0.06249 унцій;
3. 1 карат = 0.2 г=2.9412 гран;
4. 1 гран=0.068 г=0.038378 драхм;
5. 1 пайн=54.18 пек=477.33 л;
6. 1 галон (брит.)=1.2 галон (США)=4.546 л;
7. 1 галон (США)=0.0347 сак=3.785 л;
8. 1 чарка=0.0568 л=0.00012 пайна;
9. 1 квартет=291 л=5123.24 чарок;
10. 1 страйк=72.73 л=1280.46 чарок;
11. 1 челдрон=1.309 л=0.149 пека;
12. 1 сак=109 л=1.499 страйка.

Завдання 5.2 Використовуючи цикл do-while напишіть програму на C++, яка перетворює літери абетки у відповідні цифри на телефонному апараті. Програма повинна дозволяти користувачу вводити літери повторно, доки не зустрінеться літера Q або Z (літери Q і Z не використовуються в телефоні). При вводі будь-якого не абеткового символу повинно виводитися повідомлення про помилку. Між буквами і цифрами на телефонному апараті існує таке співвідношення:

ABC=2 JKL=5 TUV=8
 DEF=3 MNO=6 WXY=9
 GHI=4 PRS=7

Приклад роботи такої програми:

Введіть літеру: P

Буква P відповідає цифрі 7 на телефоні.

Введіть літеру: D

Буква D відповідає цифрі 3 на телефоні.

Введіть літеру: 2

Введений неправильний символ.

Введіть Q або Z для виходу.

Введіть літеру: Z

Вихід.

Завдання 5.3 Скласти, відлагодити та протестувати циклічну програму розв'язування задачі табулювання функції $y=f(x)$ на інтервалі $[x_{\text{поч.}} \ x_{\text{кін}}]$ з кроком h за допомогою оператора `for`. У програмі передбачати обчислення величин, вказаних у варіантах завдань.

№	$y=f(x)$	$x_{\text{поч}}$	$x_{\text{кін}}$	h	a	b	Величини, які необхідно обчислити
1	$y = \frac{e^{\sin x} + \sqrt[4]{a+x}}{\ln^3 bx}$	5.5	10.5	0.5	17.3	0.36	Середнє арифметичне $y > 5$
2	$y = \frac{\ln^4 bx + 0.85}{\sqrt[3]{a+bx^3}}$	0.4	6.8	0.8	46	1.85	Суму і кількість $y > 1$
3	$y = \frac{\sqrt[4]{1+\sqrt{ax+b}}}{\sin^2 bx + a}$	4.3	13.9	1.2	1.35	8.4	Суму $1 < y < 3$
4	$y = \frac{\operatorname{tg}^2(ax) + \sqrt{\ln x}}{e^{-bx}}$	1.3	6.1	0.6	1.8	0.56	Середнє арифметичне $y > 1$
5	$y = \frac{\sqrt[3]{a^3+x^3}}{\operatorname{tg}^3 bx + 1.6}$	0.2	1.6	0.2	1.25	0.86	Суму $0 < y < 1$
6	$y = \frac{\sqrt[3]{ax} + b}{0.25 \ln^2 ax}$	10.5	28.5	2	0.3	9.5	Добуток і кількість $y < 10$

№	$y=f(x)$	$x_{\text{поч}}$	$x_{\text{кін}}$	h	a	b	Величини, які необхідно обчислити
7	$y = \frac{\ln^2(a^3 + x^3)}{\sqrt{a^3 + x^3} + \sqrt[3]{b}}$	8.2	98.2	10	43	205	Суму і кількість $y > 0.2$
8	$y = \frac{1 + \cos^2(a^2 + x^2)}{x^3 + \sqrt[3]{bx}}$	0.5	1.9	0.2	0.84	0.63	Середнє геометричне $y > 1$
9	$y = \frac{a^x + e^{-bx}}{\sin^2 bx + 1.24}$	0.3	1.3	0.1	0.5	0.16	Добуток і суму $y > 1$
10	$y = \sqrt{\frac{ bx }{\arctg \frac{b^2}{a^2 + x^2}}}$	-10	-1	1	2.8	1.5	Середнє арифметичне $y > 0.5$
11	$y = \frac{(x^2 + 1) - \frac{1}{\sin bx}}{\sqrt[3]{\frac{x}{a}} - 0.39}$	0.2	1.6	0.1	0.36	0.74	Кількість $y > 0.5$ і добуток $y > 1$
12	$y = \frac{e^{x^2+1}}{\sqrt[5]{ x-a } + \ln^2 bx}$	1.2	3	0.2	4.8	6.8	Кількість $y > 10$ і суму $y < 10$
13	$y = \frac{e^{\sin^2 ax} + \arctg bx}{\sqrt[3]{(x+b)^2}}$	0.5	14.5	2	0.45	8.8	Кількість $y < 0.5$ і кількість $y > 0.5$
14	$y = \frac{\sin^2 ax + \sqrt[3]{ x-b }}{ x-b ^3}$	16	22	0.5	0.28	19.3	Суму і кількість $y > 2$
15	$y = \frac{\frac{a}{x^b} - \sqrt[3]{\frac{x+b}{a}}}{1.1 + \cos^2 ax}$	6.8	20.8	1	3.5	6.4	Добуток і кількість $y < 10$
16	$y = \sqrt[5]{\frac{x^2}{4a^2 + 0.6}} \ln^2(b+x)$	10	20	1	2.4	16.8	Суму і кількість $y > 20$
17	$y = \frac{\cos^2 ax + e^{-ax}}{\arctg(\sqrt{b} + \sqrt{x})}$	0.6	0.8	0.02	1.3	0.75	Кількості $y < 2.75$ і $y > 2.75$
18	$y = \frac{a - e^{bx}}{(x-a)^2 + \ln^2(x-a)}$	2	8.5	0.5	4.38	0.24	Середнє арифметичне всіх y

№	$y=f(x)$	$x_{\text{поч}}$	$x_{\text{кін}}$	h	a	b	Величини, які необхідно обчислити
19	$y = \frac{\sqrt[5]{a+x} - b}{1.6 \cos^{3.2} ax}$	0.3	3.3	0.3	1.85	2.64	Середнє геометричне $y>0$
20	$y = \frac{e^{a+x}}{\sqrt[4]{bx} + \ln^2 ax}$	0.8	2.4	0.2	1.38	9.6	Середнє арифметичне $y>5$
21	$y = \frac{\sqrt[4]{a+x} + \left(\frac{x}{b}\right)^{1.3}}{0.8 \operatorname{tg}^3 bx}$	8.5	20	0.5	17.6	0.14	Кількості $y>0$ і $y<0$
22	$y = \frac{\cos^2 x-a }{\sqrt[3]{ bx } + e^{x-a}}$	-5	5	1	1.5	14.8	Суму і кількість $y>0.1$
23	$y = \frac{1 + \cos^2(a+bx)}{0.5 \sqrt[3]{a+bx}}$	2.5	16.5	2	33.6	6.45	Середнє арифметичне $y>0.5$
24	$y = \frac{e^{\sin(x+b)} + a^3}{\ln^2(x+b)}$	0.25	7	0.5	0.86	2.8	Кількості $y<0.5$ і $y>0.5$
25	$y = \frac{\sqrt[3]{a+e^{bx}}}{a + \frac{\ln^2 x}{bx}}$	6.8	10	0.2	2.3	0.28	Суму $y>0.5$ і кількість $y>0.7$
26	$y = \frac{0.25 + \cos^2 ax}{\sqrt[4]{b^3} + x^3}$	1.3	4.3	0.3	0.43	2.24	Суму $y<0.5$ і добуток $y>0.5$
27	$y = \frac{\operatorname{arctg}(a^2 + x^2)}{\ln^3 bx + \sqrt{ax}}$	1.1	3.6	0.5	1.38	2.46	Середнє арифметичне $y>0.3$
28	$y = \frac{e^{-6x} + a^2}{\sqrt[3]{\left(\frac{a}{x}\right)^3} + \sqrt{b}}$	0.1	0.4	0.05	0.15	2.3	Добуток $y>0.4$ і суму $y>0.5$
29	$y = \frac{\sqrt{\ln a + \sqrt[3]{b^2 + 13}}}{\sin^2 ax + 0.84}$	0.04	0.24	0.02	4.2	1.54	Середнє геометричне $y>0$
30	$y = \frac{\ln^3(a+x) - 10.5}{\sqrt[4]{0.6 b^2 + \left(\frac{x}{a}\right)^2}}$	56	166	10	32	8.4	Суму і добуток $y>2$

Середнє арифметичне: $S = \frac{\sum_{i=1}^n y_i}{n}$

Середнє геометричне: $P = \sqrt[k]{\prod_{i=1}^k y_i}$

Середнє гармонійне: $G = \frac{1}{\frac{1}{n} \sum_{i=1}^n \frac{1}{y_i}}$

Зміст звіту

1. Назва та мета роботи.
2. Методика проведення роботи з графічними результатами.

Література: [1, 2, 5-7, 11].

Лабораторна робота № 6

Тема. Одновимірні масиви

Мета: навчитися використовувати одномірні масиви при розв'язуванні задач, використовувати їх в якості своєрідної бази даних, сортувати елементи одномірного масиву, шукати мінімальні і максимальні значення в масиві.

Короткі теоретичні відомості

Значення простого типу – це окрема одиниця даних і вона не може бути розбита на складові частини. Складений тип даних – це набір одиниць даних, що об'єднані одним іменем. Не зважаючи на те, що вся група має одне ім'я, можна звертатися окремо до кожного компонента. Прості типи даних є окремими блоками складених типів. Складений тип збирає набір значень і групує їх у певному порядку. Метод отримання доступу до окремого елемента структурного типу залежить від розміщення компонентів.

Нехай треба прочитати список із 1000 значень і роздрукувати його у зворотному порядку.

Довжина коду цієї програми перевищила б 3000 рядків і в ній треба було

б використати 1000 окремих змінних, якщо не використовувати складені типи. Набагато зручніше помістити цей номер у лічильник і використати цикл for від 0 до 999, а потім від 999 до 0.

```
for (number=0; number<1000; number++) cin>>value[number];  
for (number=999; number>=0; number--) cout<<value[number]<<endl;
```

Цей фрагмент коду буде працювати коректно в C++, якщо оголосити змінну value як одномірний масив. Це набір змінних одного і того ж типу в яких перша частина імені однакова, а друга частина імені значення індексу у квадратних дужках.

Оголошення одномірного масиву подібне на оголошення простого типу. Однак для масиву необхідно вказати його розмір. Це кількість компонентів у квадратних дужках

```
int value[1000];
```

Елементи масиву можуть мати практично будь-який тип. Кількість елементів у масиві має бути більше нуля. Якщо розмір масиву дорівнює n, то діапазон кількості значень від 0 до n-1.

```
float angle[4];  
int testScore[10];
```

Щоб отримати доступ до окремих компонентів масиву використовують інструкцію, що називається індексною. Вона розміщується у квадратних дужках. Ця інструкція визначає до якого компонента здійснюється доступ. Це може бути константа, ім'я змінної або складна комбінація із змінних, операторів і викликів функцій. Результат дії цієї інструкції є цілим числом, тобто одним із типів: char, short, int, long, або enum.

Для присвоєння значень елементам масиву використовується синтаксис, що наведений у прикладі

```
angle[0]=4.93;  
angle[1]=-15.2;
```

З кожним елементом масиву можна працювати так само, як і з простою змінною. Можна присвоїти йому значення, як показано вище, записати в нього

значення

```
cin>>angle[2];
```

вивести вміст цього елемента

```
cout<<angle[2];
```

використовувати його в арифметичному виразі

```
x=6.8*angle[2]+7.5;
```

C++ не здійснює перевірку значень індексів на вихід за межі масиву. Це має робити програміст. Індеси, що виходять за межі масиву мають значення менше нуля або більше розміру масиву мінус 1.

При обробці масивів часто використовують цикл `for` для переходу від одного елемента до іншого. Наведемо приклад циклу для обнулення масиву `alpha`, що складається зі 100 елементів.

```
for (i=0; i<100; i++)  
    alpha[i]=0.0;
```

Масиви можна задати за оголошення. У цьому випадку треба вказати початкові значення елементів масиву, розділяючи їх комами

```
int age[5]={23, 10, 16, 37, 12};
```

Якщо значень менше, ніж розмір масиву, то значення, котрих не вистачає, будуть дорівнювати нулю. Більша кількість значень за розмір масиву викликає синтаксичну помилку.

Як відомо, у C++ прості змінні завжди передаються за значенням. Щоб передати просту змінну за посиланням, треба до типу даних дописати символ амперсанда (&) у списку формальних параметрів функції.

У C++ неможливо передати масиви за значенням. Масиви завжди передають за посиланням. Коли масив передається як параметр функції, то в цьому випадку повертається у функцію адреса першого елемента масиву в пам'яті комп'ютера. Після цього функція знає де розміщений фактичний масив і може звертатися до будь-якого з його елементів. Наведемо приклад функції, що обнулює одномірний масив типу `float` будь-якого розміру.

```
void ZeroOut (/*out*/float arr[],/*in*/int numElements)
```



```

{
    int i;
    for (i=0; i<numElements; i++)
        arr[i]=0.0;
}

```

Найбільш поширеною помилкою при роботі з масивами є передача у функцію елемента масиву, коли необхідно передавати весь масив.

```
void Function(float[], int); //неправильна функція
```

```

...
int main()
{
    float array[50];
    ...
    Function(array, 50);
    ...
}

```

Виклик функції

Function(array[50], 50); призведе до помилки. Однак, найнебезпечніше в цьому прикладі те, що елемента масиву за номером 50 взагалі не існує. Всі елементи масиву array лежать в діапазоні від 0 до 49.

Найчастіше виконуються три види обробки масивів: використання частини оголошеного масиву, використання двох або трьох масивів і використання значень індексів, що мають специфічне значення в рамках задачі.

Розмір масиву встановлюється на етапі компіляції. В міру запису даних у масив, ми збільшуємо лічильник заповнених елементів. Під час обробки він використовується для роботи з корисними елементами. Решта не будуть розглянуті. Наприклад, якщо в класі 25 студентів, то програма для аналізу оцінок за тест буде використовувати масив із 25 елементів. Однак деякі студенти будуть відсутні в день тестування, тому буде підрахована фактична кількість оцінок. Саме ця кількість, а не значення 25, буде використана під час подальшої обробки масиву. Фактична кількість значень у масиві часто називається його довжиною. Якщо довжина масиву менша за його оголошений розмір, то саме вона повинна передаватися у функцію в якості параметра.

Наприклад:

```
void Print (/*in*/) const char grade[ ],//Масив для 25 студентів
```

*/*in*/ int length) //Фактична кількість оцінок у масиві*

У багатьох задачах потрібно об'єднати декілька частин інформації. Можна створити масив типу `int` (або `long`) для номерів і масив типу `char` для оцінок студентів. Після цього до значень масивів можна звертатися паралельно. Кожний номер відповідає певній оцінці, бо їх позиції в масивах співпадають, тобто мають рівні індекси. В деяких задачах індекс є не тільки номером позиції. Його значення має семантичний зміст. Це означає, що крім чисел і назви можуть ставитися у відповідність.

Під час компіляції програмного коду для статично оголошених масивів виділяється пам'ять. Для ефективного використання пам'яті призначене динамічне оголошення масивів, а саме

*<тип вказівника> *<назва>=new <тип змінної>[<кількість>];*

Після виконання цієї команди буде виділена неперервна ділянка пам'яті обсягом

sizeof(тип змінної)<кількість>;*

і назва масиву буде вказувати на початок цієї ділянки.

З динамічною змінною можна виконувати операції, що дозволені для даних відповідного базового типу.

Після опрацювання масиву звільнити пам'ять можна за допомогою команди

delete[]<назва вказівника на масив даних>;

Під час звільнення пам'яті розмір масиву вказувати не потрібно.

За допомогою динамічних змінних можна розв'язати завдання почергового опрацювання однією програмою деякої кількості великих масивів (якщо всі масиви неможливо одночасно ввести в пам'ять).

Хід роботи

Ввести, відлагодити та протестувати програму для розв'язування задачі згідно індивідуального варіанту.

Завдання на лабораторну роботу

Нехай прибуток фірми за k -й рік обчислюється за формулою $y_k = 100f_{i+9}(k)$ умовних одиниць, де $k=2000, 2001, \dots, 2010$; i – номер варіанта. Якщо $y_k > 0$, то вважатимемо, що фірма певного року мала прибуток, а у випадку $y_k < 0$ – збитки. Вивести на екран таблицю: номер року, величина прибутку.

Розглянути фінансову діяльність фірми протягом десяти років. Виконати додатково завдання вашого варіанта, наведене нижче. Вивести повідомлення, якщо шуканих даних немає, наприклад, якщо збитків чи прибутків не було взагалі.

1. Обчислити суму прибутків фірми. Визначити максимальний збиток (якщо збитки були).
2. Обчислити суму збитків. У якому році збиток був максимальний?
3. Обчислити суми прибутків і збитків фірми та їх різницю. Коли прибуток був максимальний?
4. Скільки років поспіль прибутків було менше, ніж 1000, але більше, ніж 500 у.о? Коли фірма зазнала найбільших збитків?
5. Обчислити суму збитків. У якому році прибуток був найбільший?
6. Обчислити суму прибутків у межах $0 < y_k < 710$ (в у.о.). У якому році фірма зазнала найбільших збитків?
7. Скільки років прибутки були в межах від 200 до 700 у.о? Які це були роки?
8. Обчислити суму всіх збитків. У якому році збиток був найбільший? Який це був збиток?
9. Обчислити суму тих збитків, для яких справджуються умови $y_k < -650$ або $y_k > -150$ (в у.о.). Визначити найбільший прибуток.
10. Визначити суми прибутків і збитків. Скільки років фірма була прибутковою?
11. Обчислити суму прибутків, що були у межах $230 < y_k < 8500$ (в у.о.). Скільки років фірма мала такі прибутки?

12. Обчислити суму збитків, що були у межах $-750 < y_k < 200$ (в у.о.). Коли дохід був мінімальний?
13. Обчислити суму прибутків і збитків за перші сім років роботи та їх різницю. Визначити максимальний прибуток за цей період.
14. Обчислити суми прибутків, що були в межах $y_k < 170$ або $y_k > 620$ (в у.о.). Скільки років фірма мала такі прибутки?
15. Обчислити суму збитків і визначити, скільки років фірма була збитковою? У якому році збиток був максимальний?
16. Визначити найбільший збиток. У якому році фірма мала найбільший прибуток?
17. У які роки фірма мала найбільші прибуток і збиток?
18. Обчислити суму збитків. Чи був хоч раз нульовий баланс?
19. Обчислити суми прибутків і збитків фірми та їх різницю. Визначити максимальний збиток фірми.
20. Обчислити суму збитків, для яких справджується умова $y_k < -590$ або $y_k > -330$ (в у.о.). Визначити найбільший прибуток і в якому році він був отриманий?
21. Обчислити суму збитків фірми. У якому році прибуток був найменший? Визначити його величину.
22. Обчислити середні арифметичні всіх прибутків і збитків.
23. Обчислити суми прибутків і збитків за перші п'ять років роботи. Скільки років протягом цього періоду фірма мала прибутки?
24. Обчислити суму прибутків, які були в межах $315 < y_k < 958$ (в у.о.). У якому році збитки були найбільші?

Зміст звіту

1. Назва та мета роботи.
2. Методика проведення роботи з графічними результатами.

Література: [1–5, 7, 8, 10, 12].

Лабораторна робота № 7

Тема. Багатовимірні масиви

Мета: навчитися використовувати багатовимірні масиви при розв'язуванні задач, використовувати їх в якості своєрідної бази даних, сортувати елементи багатовимірного масиву, шукати мінімальні і максимальні значення в масиві.

Короткі теоретичні відомості

У багатьох задачах взаємозв'язок між елементами даних є більш складним, ніж простий список. Двовірний масив використовується для представлення таблиці, що містить рядки і колонки. При цьому всі елементи мають один і той же тип даних. Щоб звернутися до окремого елемента у двовірному масиві, треба вказати його позицію в рядку і колонці.

Двовірний масив оголошується так само, як і одновірний, з тією різницею, що розміри мають вказуватись для двох вимірів:

```
const int Num_Rows=100;
const int Num_Cols=9;
...
float alpha[Num_Rows][Num_Cols];
           1-й вимір  2-й вимір
```

У цьому випадку оголошується двовірний масив alpha, елементи якого мають тип float.

Для звернення до окремого елемента масиву alpha використовуються дві інструкції (по одній для кожного виміру)

```
alpha [0] [5]=36.4;
```

Дані в рядках чи колонках можна представити ще одним способом:

```
enum DayType{MONDAY,TUESDAY,...,SUNDAY};
int hiTemp[52][7];
```

У цьому прикладі оголошений двовірний масив, але в якості індексу колонки можна використовувати інструкцію типу DayType (назви днів). Наприклад, hiTemp[2][SUNDAY] це той самий елемент, що і hiTemp[2][6]. Можна використовувати назви в індексах і по рядках і по колонках.

```
enum Colors{RED,ORANGE,YELLOW,GREEN,BLUE,INDIGO,VIOLET};
```

```
enum Brands{FORD,TOYOTA,HYUNDAI,YAGUAR,CITROEN,BMW,FIAT,SAAB};
const int NUM_COLORS=7;
const int NUM_BRANDS=8;
float crashRating[NUM_COLORS][NUM_BRANDS];
...
crashRating[BLUE][YAGUAR]=0.83;
```

Обробка даниху двомірному масиві означає звернення до масиву одним з чотирьох способів: випадковим чином, до рядків, до колонок, до цілого масиву. Кожний із способів може включати обробку підмасивів.

Випадковий доступ – коли вводиться довільна, випадкова комбінація координат. Доступ до колонок або рядків – коли над колонками чи рядками виконується певна дія. Доступ до цілого масиву – коли хочемо знайти середнє значення всіх величин масиву. В цьому випадку сумування вже йде по рядках і колонках, а не окремо. Звідси впливають способи обробки масивів:

- додавання рядків;
- додавання колонок;
- ініціалізація всіх елементів таблиці значенням 0 (або іншим);
- друк таблиці.

У прикладі нижче означимо декілька величин, які необхідні для оголошення двомірного масиву.

```
const int NUM_ROWS=50;//Кількість рядків
const int NUM_COLS=50;//Кількість колонок
int table[NUM_ROWS][NUM_COLS]; //Двомірний масив
int rowLength; //Інтервал значень від 0 до rowLength-1
int row; //Індекс для рядка
int colLength; //Інтервал значень від 0 до colLength-1
int col; //Індекс для колонок
int total; //Змінна для додавання
```

Припустимо, що потрібно додати всі елементи рядка з номером 3 у масиві table і вивести результат.

```
total=0;
for (col=0; col<NUM_COLS; col++)
    total=total+table[3][col];
cout<<"Сума рядка:"<<total<<endl;
```

Якщо треба підсумувати елементи по двох рядках, то це можна зробити за допомогою вкладених циклів.

```
total=0;
```

```
for (row=2; row<4; row++)
{
    for (col=0; col<NUM_COLS; col++)
        total=total+table[row][col];
}
cout<<"сума рядків 2 і 3:"<<total<<endl;
```

Щоб організувати додавання за колонками, то в цьому випадку потрібно у попередньому фрагменті програми замінити місцями колонки і рядки.

```
total=0;
for (col=2; col<colLength; col++)
{
    for (row=0; row<rowLength; row++)
        total=total+table[row][col];
}
cout<<"сума колонок:"<<total<<endl;
```

Двомірний масив можна ініціалізувати за оголошення або з використанням виразів присвоювання. Якщо масив невеликий, то простіше ініціалізувати його за оголошення. Наприклад, щоб ініціалізувати таблицю на 2 рядки і 3 колонки числами:

```
int table[2][3]=
{
    {14,3,-5},
    {0,46,7}
};
```

Непрактично ініціалізувати таблицю за оголошення, якщо вона велика, наприклад, 100x100. Якщо її значення різні, то краще дані зберігати у файлі і вводити в процесі роботи програми. Якщо ж усі значення однакові, то, як правило, використовуються вкладені цикли `for` і вирази присвоювання. Наведемо фрагмент коду для обнулення таблиці із `NUM_ROWS` рядків і `NUM_COLS` колонок.

```
for (row=0; row<NUM_ROWS; row++)
{
    for (col=0; col<NUM_COLS; col++)
        table [row][col]=0;
}
```

Як відомо, при використанні одномірних масивів у якості формальних параметрів функції розмір масиву, як правило, не вказується

```
void SomeFunc(*inout*/ float list[ ], /*in*/ int size)
```

Якщо вказати розмір масиву у квадратних дужках, то він буде проігнорований. У функцію передається базова адреса фактичного параметра, тобто адреса в пам'яті першого елемента масиву. Функція працює з фактичним параметром будь-якого розміру. Оскільки вона не знає розміру фактичного масиву, то їй передається значення розміру у вигляді додаткового параметра (як у прикладі з функцією `SomeFune`, що вище) або використовується константа (якщо функція завжди працює з масивами одного і того ж розміру).

Коли в якості параметра передається двовірний масив, функція також отримує базову адресу фактичного масиву. Не можна опускати розмір зразу двох вимірів. Можна не вказувати розмір першого виміру (кількість рядків), але обов'язково повинна вказуватися кількість колонок. Тобто, оголошення формального параметра має завжди містити кількість колонок

```
void AnotherFune (/*inout*/ int arr[][4]);
```

Функція `AnotherFune` працює з двовірним масивом, що має будь-яку кількість рядків, але обов'язково чотири колонки. Щоб уникнути помилок, які пов'язані з не співпаданням розмірів формальних і фактичних масивів, корисно використовувати вираз `typedef` для встановлення типу двовірного масиву. Після цього можна оголосити і фактичні і формальні параметри цього типу.

```
const int NUM_ROWS=10; const int NUM_COLS=20;
typedef int TableType[NUM_ROWS][NUM_COLS];
```

Запишемо у загальному вигляді функцію, яка ініціалізує всі елементи

масиву вказаним значенням.

```
void Initialize(/*out*/TableType table, //Масив, який треба ініціалізувати
               /*in*/int initVal //Значення для ініціалізації.)
    //Дана функція ініціалізує кожний елемент
    //таблиці значенням initVal
{
    int row;
    int col;
    for (row=0; row<NUM_ROWS; row++)
        for (col=0; col<NUM_COLS; col++)
            table[row][col]=initVal;
}
```

Після цього у коді програми можна оголосити й ініціалізувати один або

більше масивів типу `TableType` за допомогою виклику функції `Initialize`.

```
TableType delta;  
TableType gamma;  
Initialize(delta,0);  
Initialize(gamma,-1);
```

Двомірний масив можна уявити собі як масив, що складається з масивів.

Це означає, що елементи двомірного масиву не обов'язково повинні бути неподільними.

Наприклад, можна оголосити масив з імен студентів:

```
const int MAX_LENGTH=200;  
typedef char String10[11];//Місце для 10 символів і символу '\0'  
String10 student[MAX_LENGTH];//Масив із імен студентів
```

За такого оголошення елементами масиву `student` є одномірні масиви типу `String10`. Тобто масив `student` має два виміри. `student[57]` – це звернення до імені студента з номером 57. Якщо вказати два індекси `student[57][0]` – це звернення до першої літери імені студента з номером 57. Масив `student` можна оголосити і іншим способом

```
char student[MAX_LENGTH][11];
```

Два попередні способи оголошення двомірного масиву є рівноправними. Використання того чи іншого способу, буде залежати від вподобань самого програміста.

У мові C++ кількість вимірів масиву необмежена. Вимірів може бути стільки, скільки властивостей потрібно описати для кожного елемента масиву.

Хід роботи

Ввести, відлагодити та протестувати програму для розв'язування задачі згідно індивідуального варіанту.

Завдання на лабораторну роботу

Завдання 7.1 Утворити масив з елементами $a_{kn}=nf_{i+11}(k)+\sin(k)f_{i+12}(n)$, де i – номер варіанта, $k, n=1, 2, 3, 4$. Вивести його на екран у вигляді таблиці (матриці). Виконати додатково завдання вашого варіанта.

1. Визначити індекси мінімального елемента масиву. Обчислити добуток

його від'ємних елементів.

2. Обчислити кількість елементів масиву, для яких виконується нерівність $1 < a_{kn} < 6$.

3. Обчислити добуток значень тих елементів, для яких справджуються нерівності $a_{kn} < -1$ або $a_{kn} > 1$.

4. Обчислити кількість додатних елементів та їхній добуток.

5. Обчислити суму квадратів елементів, значення яких більші, ніж 1.

6. Обчислити добуток квадратів тих елементів масиву, для яких виконується нерівність $|a_{kn}| < 3$.

7. Обчислити кількість тих елементів масиву, для яких виконується нерівність $a_{kn} > 3$, та суму елементів менших, ніж 9.

8. Обчислити добуток від'ємних елементів. Визначити індекси максимального елемента.

9. Обчислити суму діагональних елементів масиву та кількість від'ємних елементів.

10. Обчислити добуток тих елементів масиву, для яких виконується нерівність $2 < a_{kn} < 10$.

11. Визначити індекси максимального елемента масиву. Обчислити добуток елементів над головною діагоналлю.

12. Обчислити добуток елементів перших двох рядків.

13. Обчислити суму елементів масиву над головною діагоналлю. Визначити індекси мінімального елемента.

14. Обчислити суму від'ємних елементів. Знайти максимальний.

15. Обчислити добуток мінімального і максимального елементів масиву.

16. Визначити індекси мінімального і максимального елементів масиву.

17. Елементи масиву, що дорівнюють нулю, замінити на 1. Знайти суму елементів під головною діагоналлю.

18. Визначити кількість від'ємних і суму додатних елементів.

19. Обчислити добуток тих елементів, для яких виконуються не рівності $a_{kn} < -5$ або $a_{kn} > 3$. Визначити індекси мінімального елемента.

20. Визначити індекси максимального та мінімального елементів масиву. Обчислити їхній добуток.
21. Обчислити добуток елементів над головною діагоналлю матриці та визначити їхню кількість.
22. Обчислити середнє арифметичне додатних елементів масиву.
23. Обчислити суму тих елементів масиву, для яких виконується нерівність $1 < a_{kp} < 5$. Знайти максимальний елемент.
24. Обчислити суму діагональних елементів матриці та кількість елементів, значення яких менші, ніж 3.
25. Обчислити добуток елементів під головною діагоналлю на суму елементів на головною діагоналлю.

Зміст звіту

1. Назва та мета роботи.
2. Методика проведення роботи з графічними результатами.

Література: [1–5, 7, 8, 10, 12].

2 КРИТЕРІЇ ОЦІНЮВАННЯ ЗНАНЬ СТУДЕНТІВ

У 2-му семестрі студенти виконують 7 лабораторних робіт. Загальна кількість балів, яку отримують студенти за виконання лабораторних робіт, становить 48 балів – сума за захист виконаних лабораторних робіт.

Шкала оцінювання: національна та ECTS

Сума балів за всі види навчальної діяльності	Оцінка ECTS	Оцінка за національною шкалою	
		Для іспиту, курсового проекту (роботи), практики	Для заліку
90–100	A	Відмінно	Зараховано
82–89	B	Добре	
74–81	C		
64–73	D	Задовільно	
60–63	E		
35–59	FX	Незадовільно з можливістю повторного складання	Не зараховано з можливістю повторного складання
0–34	F	Незадовільно з обов’язковим повторним вивченням навчальної дисципліни	Не зараховано з обов’язковим повторним вивченням навчальної дисципліни

СПИСОК ЛІТЕРАТУРИ

Основна

1. Шпак З. Я. Програмування мовою С. – Львів: Оріяна-Нова, 2006. – 432 с. іл. ISBN 5-8326-0155-6.
2. Васильєв О.М. Програмування на С++ в прикладах і задачах. Видавництво: "Ліра-К", Київ, 2017 рік, 382 с. ISBN: 978-617-7507-41-2
3. Вінник В.Ю. Алгоритмічні мови та основи програмування. Мова С. – Житомир: ЖДТК, 2007. – 328 с
4. Вступ до програмування мовою С++. Організація обчислень : навч. посіб. / Ю. А. Белов, Т. О. Карнаух, Ю. В. Коваль, А. Б. Ставровський. – К. : Видавничо-поліграфічний центр "Київський університет", 2012. – 175 с. : іл. ISBN 978-966-439-
5. Трофименко О. Г., Прокоп Ю. В., Швайко І.Г., Буката Л. М., Шаповаленко В. А., Леонов Ю. Г., Ясинський В. В.. «С++. Теорія та практика: навчальний посібник.». – 2011. – 588 с.
6. Грицюк Ю.І., Рак Т.Є Програмування мовою С++ : навчальний посібник. – Львів : Вид-во Львівського ДУ БЖД, 2011. – 292 с. ISBN 978-966-3466-85-9
7. Васильєв О.М. Алгоритми. Видавництво: "Ліра-К", Київ, 2022 рік, 424 с. ISBN: 978-617-520-353-8
8. B. Stroustrup: A Tour of C++ (Second Edition). July 2018. Addison-Wesley. ISBN 978-0-13-499783-4. 240 pages.
9. Miller R. An Introduction to the Imperative Part of C++ [Електронний ресурс] / Rob Miller, William Knottenbelt. – Режим доступу: <http://www.doc.ic.ac.uk/~wjk/C++Intro/#S1>
10. Львов М. С., Співаковський О. В. Вступ до об'єктно-орієнтованого програмування. – Херсон: ХГПУ, 2000. – 238 с.

Допоміжна

11. Bjarne Stroustrup. The C++ Programming Language (4th Edition) Addison-Wesley, May 2013.- 1368p. ISBN 978-0321563842.

12. Schildt Herbert. C++: The Complete Reference 4th edition.-McGraw-Hill Education, 2003. -1056 p.

13. Уроки програмування на C++ / aCode. – Режим доступу: <https://acode.com.ua/uroki-po-cpp/>

14. Уроки програмування на C / programiz. – Режим доступу: <https://www.programiz.com/c-programming>

15. Zagirnyak M. dComFra DC-office as a tool for digital skills development for the electrical engineering students / Zagirnyak M., Mamchur D., Gladyr A. // Proceedings of the 20th IEEE International Conference on Modern Electrical and Energy Systems, MEES 2021, 2021

Інформаційні ресурси

16. Banahan , M. The C Book — Table of Contents [Electronic resource] / Mike Banahan, Declan Brady and Mark Doran – Access mode : https://publications.gbdirect.co.uk//c_book/the_c_book.pdf [in English].

17. C Elements of Style [Electronic resource] – Access mode: <http://www.oualline.com/books.free/style/index.html> [in English].

18. Writing Bug – Free C Code [Electronic resource] – Access mode: <https://www.duckware.com/bugfreec/index.html> [in English]

Методичні вказівки щодо виконання лабораторних робіт з навчальної дисципліни «Програмування» для студентів денної форми навчання зі спеціальності 123 – «Комп’ютерна інженерія»

Укладачі: к. т. н., доц. Мамчур Д. Г.

Відповідальний за випуск гарант освітньої програми «Комп’ютерна інженерія» бакалаврського рівня А. Л. Перекрест

Підп. до др._____. Формат 60×84 1/16. Папір тип. Виведення ризографія.

Ум. Виведення. арк. _____. Наклад _____ прим. Зам. №_____.
Безкоштовно.

Видавничий відділ
Кременчуцького національного університету
імені Михайла Остроградського
вул. Першотравнева, 20, м. Кременчук, 39600